Programación 1º DAW

Teoría de los temas 1, 2 y 3 · Introducción a la POO
Ejercicios resueltos y explicados paso a paso

Índice

Empieza aquí

- Cómo usar estos apuntes en el examen
- Buscador rápido (Ctrl + F)
- Los bucles en 5 minutos
- Plantillas de examen

Parte 1. Teoría

- Tema 1. Algoritmos
- Tema 2. Java básico
- Condicionales
- Tema 3. Bucles
- Introducción a la POO
- Patrones que se repiten en todos los ejercicios
- Chuleta para el examen

Parte 2. Ejercicios explicados

- UD1. Algoritmos y expresiones (en papel)
- UD2 P1. Condicionales básicos (2.1 a 2.14)
- UD2 P2. Condicionales y aleatorios (2.11 a 2.19)
- Tema 2. Ejercicios adicionales (24 a 26)
- UD3 I. Bucles básicos (3.1 a 3.15)
- UD3 II. Bucles con cuentas (3.11 a 3.20)
- Tema 3. Ejercicios adicionales (26 a 33)

Parte 3. Exámenes de otros años resueltos

- Examen práctico Temas 1, 2 y 3 (08-10-2025)
- Examen Tema 3 (22-11-2024)
- Examen Temas 2 y 3 (08-11-2024)

Parte 4. Otros ejercicios de clase

- Triaje hospitalario
- Juego de los dados
- Árboles de Navidad

Empieza aquí

Cómo usar estos apuntes en el examen

1. **Lee el enunciado y subraya qué pide.** Busca palabras como "hasta que", "entre X e Y", "menú", "dibuja", "cifras", "primo", "aleatorio".
2. **Mira la tabla «Buscador rápido»** de la página siguiente y busca la palabra clave con **Ctrl + F** (todas empiezan por `#`, por ejemplo `#capicua`). Te lleva directo al ejercicio o a la plantilla que se parece.
3. **Copia el código del ejercicio parecido** y cambia solo lo necesario:
   - el nombre de la clase (tiene que ser igual que el del archivo),
   - los mensajes de `System.out.println`,
   - los números y las condiciones del enunciado.
4. **Compila.** Si no compila, mira la sección `#errores` al final de la teoría.
5. **Prueba con los ejemplos del enunciado** y con un caso raro (0, un negativo, un número fuera de rango).

Consejo: un ejercicio que compila y hace casi todo puntúa; uno que no compila, no. Primero que compile y después lo mejoras.

Buscador rápido (Ctrl + F)

Cada ejercicio y plantilla lleva una línea **Palabras clave** con etiquetas que empiezan por `#`. Busca la etiqueta tal cual (sin tildes).

Si el examen pide…	Busca	Dónde está el más parecido
Par / impar	`#par`	2.1, 3.1
Mayor de dos o tres números, ordenar	`#mayor` `#ordenar`	2.3, 2.6, 2.7
Rango / "entre X e Y" / casi cero	`#rango`	2.4, 2.9
Ecuación de segundo grado / raíz	`#ecuacion` `#raiz`	2.8, 3.19
Número en texto / nota en texto / día de la semana	`#switch` `#texto`	2.10, 2.11, 2.14, examen dígitos
Fechas (válida, día siguiente)	`#fecha`	2.12, 2.14
Horas (segundo siguiente, sumar segundos, minutos y segundos)	`#hora`	2.13, 3.13, ej. 26 MostrarTiempo
Cifras de un número / darle la vuelta / capicúa	`#cifras` `#capicua`	2.9, 2.11, examen dígitos
Letra del DNI	`#dni`	2.12 (UD2 P2)
División entre cero / repartir	`#dividir-cero`	2.13 (granja)
Números aleatorios / juegos de adivinar	`#aleatorio` `#juego`	2.17, 2.18, 3.3, 3.5, dados
Valor absoluto / ternario	`#ternario`	2.16

Si el examen pide…	Busca	Dónde está el más parecido
Grados, millas, temperaturas (conversiones)	`#conversion`	2.19, 25, examen temperaturas
Pedir hasta que meta 0 / -1 / negativo	`#centinela`	3.1, 3.2, 3.4, 3.20, IRPF
Pedir hasta que el dato sea válido	`#validar`	3.9, examen movimiento
Contar cuántos cumplen algo	`#contador`	3.2, 3.13, ej. 24
Sumas y medias	`#media` `#acumulador`	3.2, 3.8, ej. 24
El mayor / el más alto / máximo	`#maximo`	3.4, dados
¿Hay alguno que…? (bandera)	`#bandera`	3.12, primos
Factorial / potencias	`#factorial`	3.11
Múltiplos / divisores	`#multiplos`	3.7, examen múltiplos
Tablas de multiplicar	`#tabla-multiplicar`	3.9, 3.14
Primos	`#primos`	3.14, ej. 26
MCD / MCM	`#mcd` `#mcm`	3.17, 3.18
Binario / hexadecimal	`#binario` `#hexadecimal`	3.11, 3.12, ej. 32, 33
Triángulo de Pascal	`#pascal`	3.15
Triángulos de asteriscos	`#triangulo`	3.15, 3.16, examen triángulos
Cuadrados (llenos, huecos, O digital)	`#cuadrado`	ej. 27, 28, 29
Rombos	`#rombo`	ej. 30, 31
Árbol de Navidad / reloj de arena	`#arbol` `#reloj-arena`	exámenes, extras
Matriz / tabla de números	`#matriz`	examen matriz formateada, examen matriz +/-
Cuadrado de números que sube y baja	`#cuadrado-numeros`	examen cuadrado
Menú de opciones	`#menu`	examen encriptación
Textos: cambiar letras, `charAt`	`#texto` `#charAt`	examen encriptación, 2.12, ej. 33
Clasificar por tramos (IRPF, triaje, notas)	`#tramos`	IRPF, triaje, 2.10
Varios jugadores / ganador / empate	`#jugadores` `#empate`	dados
Plantillas listas para copiar	`#plantilla`	Plantillas de examen

Los bucles en 5 minutos

Un **bucle** repite un trozo de código para no tener que escribirlo muchas veces. Solo hay que elegir cuál:

Pregunta	Respuesta	Bucle
¿Sé cuántas vueltas son? ("10 números", "del 1 al n")	Sí	`for`

Pregunta	Respuesta	Bucle
¿Puede que no haga ninguna vuelta? ("hasta que meta 0")	Sí	`while`
¿Tiene que hacerlo al menos una vez? ("pide hasta que sea válido", menú)	Sí	`do-while`

while: "mientras queden galletas, como"

Antes de cada vuelta **mira** la condición. Si ya es falsa al llegar, no entra ni una vez.

```
int galletas = 3;
while (galletas > 0) {          // ¿quedan? -> sí -> entro
    System.out.println("Me como una galleta");
    galletas--;                 // IMPORTANTE: algo tiene que cambiar o no para nunca
}
System.out.println("No quedan");
```

Sale: "Me como una galleta" ×3 y "No quedan". Si `galletas` empezara en 0, solo saldría "No quedan".

do-while: "pedir el PIN"

Primero **hace** y después mira si repite. Siempre hace al menos una vuelta.

```
int pin;
do {
    System.out.print("PIN: ");
    pin = sc.nextInt();
} while (pin != 1234);          // se repite MIENTRAS esté MAL. Lleva ; al final
System.out.println("Desbloqueado");
```

for: "haz 10 flexiones"

Sabes desde el principio cuántas vueltas son y vas contando.

```
for (int i = 1; i <= 10; i++) {
//   inicio   condición  paso
    System.out.println("Flexión " + i);
}
```

- **Inicio** `int i = 1`: se hace una vez.
- **Condición** `i <= 10`: se mira antes de cada vuelta.
- **Paso** `i++`: se hace al final de cada vuelta (suma 1).

`i` es solo un nombre (de "índice"); puede llamarse `numero`, `fila`... Si empiezas en 0 usa `<`; si empiezas en 1 usa `<=`.

for dentro de for: "un reloj"

Por cada vuelta del de fuera, el de dentro se recorre entero. El de fuera son las **filas**, el de dentro lo que va **en cada fila**. Así se hacen todas las figuras, tablas y matrices.

```
for (int fila = 1; fila <= 3; fila++) {
    for (int col = 1; col <= 4; col++) {
        System.out.print("*");      // print: sin saltar de línea
    }
    System.out.println();           // al acabar la fila, salto
}
```

Sale 3 filas de `****`.

Un for es un while escrito corto

```
for (int i = 0; i < 5; i++) { ... }

int i = 0;           // inicio, antes
while (i < 5) {      // condición
    ...
    i++;             // paso, al final de dentro
}
```

Plantillas de examen

Palabras clave: `#plantilla`

Copia la que necesites **dentro del** `main`, después de `Scanner sc = new Scanner(System.in);`, y cambia nombres, mensajes y condiciones. Todas compilan tal cual si declaras las variables que usan.

Plantilla: programa vacío

Palabras clave: `#plantilla-base` `#esqueleto`

```
import java.util.Scanner;

public class NombreDelArchivo {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        // 1. variables
        // 2. pedir datos
        // 3. comprobar que son válidos
        // 4. calcular
        // 5. mostrar
        sc.close();
    }
}
```

Plantilla: pedir hasta que metan un valor de salida

Palabras clave: `#plantilla-centinela` `#centinela` `#hasta-que`

```
int numero;
System.out.print("Número (0 para salir): ");
numero = sc.nextInt();
while (numero != 0) {              // cambia el 0 por la condición de salida
    // ... hacer cosas con numero ...
    System.out.print("Número (0 para salir): ");
    numero = sc.nextInt();
}
```

Plantilla: pedir un dato válido

Palabras clave: `#plantilla-validar` `#validar` `#rango`

```
int dato;
do {
```

```
    System.out.print("Número entre 1 y 10: ");
    dato = sc.nextInt();
    if (dato < 1 || dato > 10) {
        System.out.println("Error: tiene que estar entre 1 y 10");
    }
} while (dato < 1 || dato > 10);    // la condición de "está MAL"
```

Plantilla: contar, sumar, media y máximo

Palabras clave: `#plantilla-estadistica` `#contador` `#acumulador` `#media` `#maximo`

```
int cantidad = 0, suma = 0, cumplen = 0, maximo = Integer.MIN_VALUE;
for (int i = 1; i <= 5; i++) {      // o un while con centinela
    System.out.print("Valor " + i + ": ");
    int valor = sc.nextInt();
    cantidad++;                     // contador
    suma = suma + valor;            // acumulador
    if (valor >= 18) {              // contador con condición
        cumplen++;
    }
    if (valor > maximo) {           // máximo
        maximo = valor;
    }
}
if (cantidad > 0) {
    System.out.println("Media: " + (double) suma / cantidad);
}
```

Plantilla: ¿hay alguno que…? (bandera)

Palabras clave: `#plantilla-bandera` `#bandera`

```
boolean hayAlguno = false;
for (int i = 1; i <= 5; i++) {
    int valor = sc.nextInt();
    if (valor < 5) {
        hayAlguno = true;
    }
}
if (hayAlguno) { ... } else { ... }
```

Plantilla: recorrer las cifras de un número

Palabras clave: `#plantilla-cifras` `#cifras` `#capicua`

```
int copia = numero, cifras = 0, sumaCifras = 0, alReves = 0;
while (copia > 0) {
    int cifra = copia % 10;         // última cifra
    cifras++;
    sumaCifras = sumaCifras + cifra;
    alReves = alReves * 10 + cifra; // el número al revés
    copia = copia / 10;             // quito la última cifra
}
```

Plantilla: menú con switch

Palabras clave: `#plantilla-menu` `#menu` `#switch`

```
boolean salir = false;
while (!salir) {
    System.out.println("1. Opción uno");
```

```
    System.out.println("2. Opción dos");
    System.out.println("Otra cosa: salir");
    String opcion = sc.nextLine();  // como texto: si meten una letra no casca
    switch (opcion) {
        case "1":
            // ...
            break;
        case "2":
            // ...
            break;
        default:
            System.out.println("Fin del programa");
            salir = true;
    }
}
```

Plantilla: número aleatorio

Palabras clave: `#plantilla-aleatorio` `#aleatorio` `#random`

```
int dado = (int) (Math.random() * 6) + 1;       // entre 1 y 6
int n = (int) (Math.random() * 100) + 1;        // entre 1 y 100
int op = (int) (Math.random() * 3);             // 0, 1 o 2
```

Plantilla: ¿es primo?

Palabras clave: `#plantilla-primo` `#primos`

```
boolean esPrimo = numero > 1;
for (int d = 2; d * d <= numero && esPrimo; d++) {
    if (numero % d == 0) {
        esPrimo = false;
    }
}
```

Plantilla: figura con asteriscos

Palabras clave: `#plantilla-figura` `#triangulo` `#arbol` `#figura`

```
for (int fila = 0; fila < alto; fila++) {
    for (int i = 0; i < alto - 1 - fila; i++) {   // relleno izquierda
        System.out.print(" ");
    }
    for (int i = 0; i < fila * 2 + 1; i++) {      // asteriscos: 1, 3, 5...
        System.out.print("*");
    }
    System.out.println();
}
```

Para cualquier figura: dibújala en papel, apunta para cada fila cuántos espacios y cuántos `*` lleva y saca la fórmula (fila 0 → 1, fila 1 → 3, fila 2 → 5 = `fila * 2 + 1`).

Plantilla: matriz de números

Palabras clave: `#plantilla-matriz` `#matriz`

```
for (int fila = 1; fila <= alto; fila++) {
    for (int col = 1; col <= ancho; col++) {
        int valor = fila * col;            // la fórmula del enunciado
        System.out.print(valor + "\t");    // \t = tabulador, queda alineado
```

```
    }
    System.out.println();
}
```

Plantilla: intercambiar dos variables

Palabras clave: `#plantilla-intercambio` `#ordenar`

```
int aux = a;
a = b;
b = aux;
```

Parte 1. Teoría

Un programa es una lista de pasos que **lee datos, hace cosas con ellos y saca un resultado**. Todo lo que viene (variables, `if`, bucles) son herramientas para escribir esos pasos.

Tema 1. Algoritmos

Palabras clave: `#algoritmo` `#pseudocodigo`

Un **algoritmo** es una receta: una lista de pasos claros, sin dudas, que resuelve un problema concreto y que **termina**. Se sigue de forma mecánica, paso a paso.

Elemento	Qué es	En el ordenador
Procesador	El que entiende y ejecuta los pasos	La CPU
Entorno	El "material" con el que trabaja	Los datos en memoria
Acción	Cada cosa que hace y que cambia el entorno	Sumar, restar, copiar…

Formas de escribir un algoritmo:

Forma	Cómo es	Pega
Lenguaje natural	Como lo explicarías hablando	Es ambiguo y depende del idioma
Organigrama (diagrama de flujo)	Dibujo con cajas: inicio/fin, entrada, salida, decisión (rombo), proceso	Tedioso; el ordenador no lo entiende
Pseudocódigo	Medio idioma, medio código ("leer", "escribir", "si… entonces")	Solo sirve para personas
Lenguaje de programación	Java, FORTRAN…	Lo entienden personas y ordenador

Fases para resolver un problema:

1. **Análisis (el QUÉ):** entender el problema. ¿Qué datos entran, qué tiene que salir, qué errores puede haber? Es lo que hacemos con los comentarios `//entrada`, `//proceso`, `//salida`.
2. **Diseño (el CÓMO):** partir el problema en trozos más pequeños, y esos en otros más pequeños, hasta que cada trozo sea fácil de programar.

Conceptos que salen en preguntas:

- **Variable:** una caja con nombre que guarda un valor que puede cambiar.
- **Estado:** los valores de todas las variables en un momento dado.
- **Constante:** variable que no cambia nunca. Se escribe en MAYÚSCULAS (`PI`).
- **Literal:** un valor escrito tal cual: `45`, `3.14`, `'E'`, `"hola"`.
- **Expresión:** combinación de variables, valores y operadores: `2 * PI * r`.
- **Asignación:** guardar un valor en una variable. En pseudocódigo `edad ← 18`, en Java `edad = 18;`.

Nombres de variables: letras, números y `_`, pero **no** pueden empezar por número ni tener espacios, ñ o tildes. Válidos: `v1`, `b_n`, `aceleracion`. No válidos: `1n`, `año`, `aceleración`, `p v`. El nombre tiene que decir lo que guarda (`dni` mejor que `a`).

Tema 2. Java básico

Palabras clave: `#tipos` `#operadores` `#scanner` `#casting`

Java es **multiplataforma**: el mismo programa funciona en Windows, Linux o Mac.

1. Escribes el código en un archivo `.java`.
2. `javac` lo compila a **bytecode** (archivo `.class`, que no es lenguaje máquina).
3. La **máquina virtual de Java (JVM/JRE)** de cada sistema lee ese `.class` y lo ejecuta.

No tiene punteros, es orientado a objetos, maneja errores con excepciones y distingue mayúsculas de minúsculas (`nota` y `Nota` son variables distintas). Ediciones: **SE** (escritorio), **EE** (servidores) y **ME** (móviles y aparatos pequeños).

Estructura de cualquier programa

```java
import java.util.Scanner;           // para poder leer del teclado

public class Ejemplo {              // mismo nombre que el archivo Ejemplo.java
    public static void main(String[] args) {   // aquí empieza el programa
        Scanner sc = new Scanner(System.in);
        int edad;                     // declarar: tipo + nombre
        System.out.print("Edad: ");   // escribir sin saltar de línea
        edad = sc.nextInt();          // leer un entero
        System.out.println("Tienes " + edad);  // escribir y saltar de línea
        sc.close();
    }
}
```

Tipos de variables más usados

Tipo	Guarda	Ejemplo
`int`	Números enteros	`int edad = 18;`
`double`	Números con decimales	`double nota = 7.5;`
`boolean`	`true` o `false`	`boolean aprobado = true;`
`char`	Un solo carácter, comillas simples	`char letra = 'A';`
`String`	Texto, comillas dobles	`String nombre = "Marcos";`
`long`	Enteros muy grandes	`long factorial = 1;`

Con `final` delante se hace una constante: `final double PI = 3.1416;`. Si un tipo se pasa de su máximo, "da la vuelta" (un `byte` con 127 + 1 pasa a -128). En los apuntes se recomienda poner delante del nombre una letra con el tipo: `iContador`, `dNota`, `bAprobado`, `strNombre`.

Leer del teclado (Scanner)

Método	Lee
`sc.nextInt()`	Un entero

Método	Lee
`sc.nextDouble()`	Un número con decimales
`sc.nextLine()`	Toda la línea hasta pulsar Intro
`sc.next()`	Una palabra (hasta un espacio)

Operadores

Tipo	Operadores	Para qué
Aritméticos	`+ - * / %`	Cuentas. `%` da el **resto** de la división (`7 % 2` = 1)
Incremento	`++ --`	Sumar o restar 1 (`i++`)
Asignación	`= += -= *= /= %=`	`x += 3` es lo mismo que `x = x + 3`
Relacionales	`> >= < <= == !=`	Comparar. Dan `true` o `false`
Lógicos	`&&` (y), `||` (o), `!` (no)	Juntar condiciones
Ternario	`condicion ? a : b`	Un if-else en una línea

- `=` guarda un valor; `==` compara.
- `+` con texto **une**: `"Tienes " + 18` da `"Tienes 18"`.
- **División entera:** entre dos `int`, `/` quita los decimales: `7 / 2` da `3`. Para tener `3.5`: `7 / 2.0` o `(double) 7 / 2`.
- **Operadores de bits** (`& | ^ << >>`): trabajan con el número en binario. `10 & 12` → 1010 y 1100 = 1000 = 8. A binario se pasa dividiendo entre 2 y leyendo los restos de abajo arriba (14 = 1110).

Conversión de tipos (casting)

- **Implícita:** Java la hace solo cuando el tipo pequeño cabe en el grande: `double d = 3;` da `3.0`.
- **Explícita:** la haces tú poniendo el tipo entre paréntesis cuando puedes perder datos: `int n = (int) 3.9;` da `3`. Es lo que usamos en `(int) (Math.random() * 6) + 1`.

Comentarios

`// una línea`, `/* varias líneas */` y `/** documentación JavaDoc */` (con `@author`, `@version`...).

Condicionales

Palabras clave: `#if` `#switch` `#ternario`

Un condicional hace que el programa **elija un camino**: si la condición es `true` hace una cosa, si es `false` otra (o nada).

Tipo	Cuándo usarlo
`if`	Solo hay que hacer algo cuando se cumple la condición
`if - else`	Dos caminos: o uno o el otro

Tipo	Cuándo usarlo
`if - else if - else`	Varios casos seguidos (tramos, notas, colores)
`switch`	Una variable que puede valer varios valores concretos (menú, día de la semana)
Ternario `? :`	Un if-else que solo guarda un valor en una variable

if / else if / else

```java
if (nota >= 9) {
    System.out.println("Sobresaliente");
} else if (nota >= 5) {
    System.out.println("Aprobado");
} else {
    System.out.println("Suspenso");
}
```

Java mira las condiciones **de arriba abajo** y entra solo en la **primera** que se cumple. Por eso el orden importa: en el triaje se pregunta primero por el rojo (lo más grave), luego el amarillo y si no, verde. Condiciones compuestas: `nota >= 5 && nota < 7` (las dos a la vez) o `fc < 40 || fc > 110` (con una vale).

switch

```java
switch (opcion) {
    case 1:
        System.out.println("Uno");
        break;          // sin break se ejecutan también los case de abajo
    case 2:
    case 3:             // 2 y 3 hacen lo mismo
        System.out.println("Dos o tres");
        break;
    default:            // si no coincide ningún case
        System.out.println("Otra cosa");
}
```

Forma corta con flecha (sin `break`, solo si cada case tiene una instrucción):

```java
switch (nota) {
    case 0, 1, 2, 3, 4 -> System.out.println("Suspenso");
    case 5 -> System.out.println("Suficiente");
    default -> System.out.println("Nota no válida");
}
```

Operador ternario

```java
boolean par = numero % 2 == 0 ? true : false;
double absoluto = num < 0 ? -num : num;
```

Se lee: ¿se cumple la condición? Si sí, el valor de antes de los `:`; si no, el de después.

Tema 3. Bucles

Palabras clave: `#bucles` `#while` `#for` `#do-while`

Un bucle **repite** un trozo de código mientras se cumpla una condición. Hay dos familias: los que dependen de una **condición** (`while`, `do-while`) y los que dependen de un **contador** (`for`).

Bucle	Cuántas veces se ejecuta	Úsalo cuando…
`while`	De 0 a N (mira la condición **antes**)	No sabes cuántas vueltas: "hasta que metan un 0"
`do-while`	De 1 a N (mira la condición **después**)	Al menos una vez: pedir un dato hasta que sea válido, un menú
`for`	Las que marque el contador	Sabes cuántas vueltas: "pide 10 números", "del 1 al n"

Los ejemplos fáciles de cada bucle (galletas, PIN, flexiones, reloj) están en **«Los bucles en 5 minutos»**, al principio.

Truco para elegir: ¿sabes cuántas vueltas? → `for`. ¿No lo sabes y puede que ninguna? → `while`. ¿No lo sabes pero al menos una seguro? → `do-while`.

Cualquier `for` se puede escribir como `while`: el inicio va antes del bucle, la condición en el `while` y el `i++` al final de dentro.

Bucles infinitos y banderas

Un **bucle infinito** es uno cuya condición **nunca** llega a ser `false`: `for (int i = 0; i >= 0; i++)`, `while (true)` o un `while` donde se olvida volver a leer el dato o hacer el `i++`. El programa se cuelga.

Una **bandera** es un `boolean` que avisa de que ha pasado algo: `salir = true` en un menú, `haySuspenso = true` si una nota es menor que 5, `esPrimo = false` al encontrar un divisor.

Introducción a la POO

Palabras clave: `#poo` `#clase` `#objeto`

La **Programación Orientada a Objetos** organiza el programa en "cosas" (objetos) parecidas a las del mundo real.

Concepto	Qué es	Ejemplo
Clase	El molde: qué datos y acciones tendrán los objetos	La clase `Coche`
Objeto (instancia)	Una cosa concreta hecha con ese molde	El coche 1489-CFG, blanco, Renault Megane
Mensaje	Un objeto le pide a otro que haga algo (llama a un método suyo)	`impresora.imprimirDocumento("Hola", color)`

- **Estado → atributos:** los datos que lo describen (matrícula, color, puertas).
- **Comportamiento → métodos:** lo que sabe hacer (arrancar, acelerar, frenar).
- Pueden ser **públicos** (cualquiera los usa) o **privados** (solo el propio objeto; por ejemplo el `saldo` de una cuenta).

Un mensaje tiene tres partes: `objetoReceptor.metodo(parametro1, parametro2);`. Ya lo usáis: en `sc.nextInt()`, `sc` es un objeto de la clase `Scanner` y `nextInt()` es su método.

Patrones que se repiten en todos los ejercicios

Patrón	Para qué	Cómo
Centinela	"Hasta que meta 0 / -1 / un negativo"	Leer antes del `while` y otra vez al final de dentro
Validar	"Asegúrate de que está en el rango"	`do { pedir } while (está MAL);`
Contador	Cuántos cumplen algo	Empieza en 0, `contador++`
Acumulador	Sumas y medias	Empieza en 0, `suma = suma + valor`; media fuera del bucle
Producto	Factorial	Empieza en 1, `producto = producto * i`
Máximo	El mayor de una serie	`if (valor > max) max = valor;`
Bandera	¿Ha pasado algo alguna vez?	`boolean` que empieza en `false` y se pone a `true`
Cifras	Trabajar con cada cifra	`num % 10` = última cifra; `num / 10` = quitarla
Aleatorio	Número entre 1 y N	`(int) (Math.random() * N) + 1`
Figuras	Dibujos con `*`	`for` de filas + `for` de lo que va en cada fila; `print` dentro, `println()` al final

Chuleta para el examen

Palabras clave: `#errores` `#chuleta` `#no-compila`

El ejercicio que **no compila no cuenta**. Antes de entregar:

- El nombre del archivo es igual que el de la clase.
- Está el `import java.util.Scanner;`.
- Cada `{` tiene su `}` y cada instrucción acaba en `;`.
- Comparo con `==`, no con `=`. Los `String` se comparan con `.equals()`.
- El do-while lleva `;` al final: `} while (condicion);`.
- Cada `case` del switch lleva su `break`.
- En los `while`, algo dentro cambia la condición (vuelvo a leer o hago `i++`).
- Los límites están bien: "entre 10 y 100 incluidos" es `>= 10 && <= 100`.
- Si divido enteros y quiero decimales, uso `double` o `(double)`.
- Lo he probado con los ejemplos del enunciado y con un caso raro (0, negativo, fuera de rango).

Error típico	Qué pasa	Solución
`nextInt()` y luego `nextLine()`	El `nextLine()` se "come" el Intro y lee vacío	Un `sc.nextLine();` extra después del `nextInt()`
Confundir `print` y `println` en figuras	Todo en una columna o en una línea	`print` dentro de la fila, `println()` al final

Error típico	Qué pasa	Solución
`\` sola en un texto	Error de compilación	Escribirla doble: `"\\"`
Variable sin valor inicial	"might not have been initialized"	Darle valor al declararla (`int suma = 0;`)
Contador que no se reinicia	Se acumula entre jugadores o filas	Ponerlo a 0 al principio de cada vuelta

En papel: escribe primero en comentarios la entrada, el proceso y la salida. Luego pasa cada paso a código.

Parte 2. Ejercicios explicados

Cada ejercicio tiene: **qué pide**, **la idea** para resolverlo, los **pasos** cuando hacen falta, **cómo cambiarlo** si te piden una variante y el **código completo** (el mismo que el `.java` del mismo nombre).

Cómo estudiarlo: lee la idea, intenta hacerlo sin mirar el código y, si te atascas, mira los pasos. Al final compara con el código.

UD1. Algoritmos y expresiones (en papel)

Dos reglas lo resuelven casi todo:

- `X = Y` significa "copia el valor de Y en X". Lo de la derecha se calcula primero con los valores **actuales** y luego se guarda a la izquierda.
- Orden de operaciones: paréntesis → potencias (`^`) → `*` `/` `%` (de izquierda a derecha) → `+` `-` → comparaciones → `NOT` → `AND` → `OR`.

Ejercicio 1. Tabla de variables

Palabras clave: `#tabla-variables` `#traza`

Qué pide: A=4, B=2, C=3. Apuntar cómo quedan tras cada instrucción. En cada fila solo cambia la variable de la izquierda del `=`; las otras se copian de la fila de arriba.

#	Instrucción	Cuenta	A	B	C
—	Inicio	—	4	2	3
1	A = B	A toma el 2 de B	2	2	3
2	C = A	C toma el 2 de A	2	2	2
3	B = (A + B + C) / 2	(2+2+2)/2 = 3	2	3	2
4	A = A + C	2 + 2 = 4	4	3	2
5	C = B - A	3 - 4 = -1	4	3	-1
6	C = C - A	-1 - 4 = -5	4	3	-5
7	A = A * B	4 · 3 = 12	12	3	-5
8	A = A + 3	12 + 3 = 15	15	3	-5
9	A = A % B	15 % 3 = 0	0	3	-5
10	C = C + A	-5 + 0 = -5	0	3	-5

Resultado final: A = 0, B = 3, C = -5. **Trampa:** en la fila 5 se usa el A de la fila 4 (4), no el del principio.

Ejercicio 2. Evaluar expresiones

Palabras clave: `#expresiones` `#evaluar`

`^` es potencia. x = 1, y = 4, z = 10, pi = 3.14, e = 2.71.

Expresión	Pasos	Resultado
((3 + 2) ^ 2 – 15) / 2 * 5	(25 – 15) = 10 → 10 / 2 = 5 → 5 · 5	**25**
5 – 2 > 4 AND NOT 0.5 == 1 / 2	3 > 4 es falso → falso AND …	**falso**
2 * x + 0.5 + y – 1 / 5 * z	2 + 0.5 + 4 – (0.2 · 10)	**4.5**
pi * x ^ 2 > y OR 2 * pi * x <= z	3.14 > 4 falso; 6.28 <= 10 verdadero	**verdadero**
e ^ (x – 1) / (x * z) / (x / z)	e^0 = 1 → 1 / 10 = 0.1 → 0.1 / 0.1	**1**
"Don " + "Juan" == "Don Juan" OR "A" == "a"	verdadero OR falso	**verdadero**

Truco: con AND, si una parte es falsa todo es falso; con OR, si una parte es verdadera todo es verdadero.

Ejercicio 3. Algoritmo para cambiar una rueda

Palabras clave: `#algoritmo` `#rueda`

Pasos claros, en orden, y que terminen:

1. Aparcar en un sitio seguro, freno de mano e intermitentes.
2. Chaleco y triángulos.
3. Sacar la rueda de repuesto, el gato y la llave.
4. Aflojar un poco los tornillos de la rueda pinchada (sin quitarlos).
5. Colocar el gato y levantar el coche hasta que la rueda no toque el suelo.
6. Quitar los tornillos y la rueda pinchada.
7. Poner la de repuesto y apretar los tornillos a mano.
8. Bajar el coche y quitar el gato.
9. Apretar bien los tornillos en cruz.
10. Guardar la rueda pinchada y las herramientas. Fin.

Lo que valoran: una acción por paso, un orden con sentido (no se aflojan tornillos con el coche levantado porque la rueda gira) y un final.

Ejercicio 4. Algoritmo para cocinar pasta

Palabras clave: `#algoritmo` `#pasta`

Se puede añadir una **decisión** y una **repetición** (lo que luego serán el `if` y el bucle):

1. Llenar una olla de agua y ponerla al fuego.
2. **Mientras** el agua no hierva, esperar.
3. Echar sal y la pasta.
4. Esperar el tiempo del paquete, removiendo de vez en cuando.
5. Probar la pasta. **Si** está dura, esperar 1 minuto y volver al paso 5.
6. Escurrir, servir y añadir la salsa. Fin.

Ejercicio 5. Variable y constante

Palabras clave: `#variable` `#constante`

- **Variable:** valor que **puede cambiar** mientras el programa funciona. Ejemplos: tu edad, el saldo de la cuenta, la temperatura, los puntos de un partido.

- **Constante:** valor que **no cambia nunca**. Ejemplos: PI, los días de la semana (7), los minutos de una hora (60).

En Java: `final double PI = 3.1416;`. Si intentas cambiarla, no compila.

Ejercicio 6. Más expresiones

Palabras clave: `#expresiones` `#evaluar` `#division-entera`

OJO: en Java `/` entre dos enteros quita los decimales (7 / 2 = 3). Con números reales sería 3.5. Pongo las dos cuando cambia.

#	Expresión	Pasos	Resultado
1	24 % 5	24 = 5·4 + 4	**4**
2	7 / 2 + 2.5	Java 3 + 2.5 / real 3.5 + 2.5	**5.5** (Java) / **6** (real)
3	10.8 / 2 + 2	5.4 + 2	**7.4**
4	(4 + 6) * 3 + 2 * (5 - 1)	30 + 8	**38**
5	5 / 2 + 17 % 3	Java 2 + 2 / real 2.5 + 2	**4** (Java) / **4.5** (real)
6	7 >= 5 OR 27 <> 8	V OR V	**verdadero**
7	(45 <= 7) OR NOT (5 >= 7)	F OR NOT F = F OR V	**verdadero**
8	27 % 4 + 15 / 4	Java 3 + 3 / real 3 + 3.75	**6** (Java) / **6.75** (real)
9	37 / 4 * 4 – 2	Java 9·4 – 2 / real 9.25·4 – 2	**34** (Java) / **35** (real)
10	(25 >= 7) AND NOT (7 <= 2)	V AND NOT F	**verdadero**
11	('H' < 'J') AND ('9' <> '7')	H va antes que J; 9 ≠ 7	**verdadero**
12	25 > 20 AND 13 > 5	V AND V	**verdadero**
13	10 + 4 < 15 - 3 OR 2 * 5 + 1 > 14 – 2 * 2	14 < 12 F; 11 > 10 V	**verdadero**
14	4 * 2 <= 8 OR 2 * 2 < 5 AND 4 > 3 + 1	8 <= 8 V → OR ya es V	**verdadero**
15	10 <= 2 * 5 AND 3 < 4 OR NOT (8 > 7) AND 3 * 2 <= 4 * 2 - 1	(V AND V) OR (F AND …)	**verdadero**

`<>` significa "distinto" (en Java `!=`). Las letras se comparan por orden alfabético.

Ejercicio 7. Ordinograma

Palabras clave: `#ordinograma` `#area-circulo`

Lee R (el radio), calcula A = 3.14 · R · R y lo muestra: es el **área de un círculo**. Con R = 2: A = 3.14 · 2 · 2 = **12.56**.

```
radio = sc.nextDouble();
area = 3.14 * radio * radio;
System.out.println(area);
```

UD2 P1. Condicionales básicos (2.1 a 2.14)

Todos son **pedir datos → un `if` o `switch` → mostrar**. Lo importante es elegir bien la condición y el orden de los casos.

2.1 Par o impar

Palabras clave: `#par` `#impar` `#resto`

Idea: un número es par si al dividirlo entre 2 no sobra nada. `%` da lo que sobra: `numero % 2 == 0`. **Para cambiarlo:** ¿múltiplo de 3? `numero % 3 == 0`.

Código completo · `Ej2_1_ParImpar.java`

```java
import java.util.Scanner;

/*
 * 2.1 Diseñar una aplicación que solicite al usuario un número e indique si es par o impar.
 *
 * Idea: un número es par si al dividirlo entre 2 el resto (%) es 0.
 */
public class Ej2_1_ParImpar {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numero;

        System.out.print("Introduce un número: ");
        numero = sc.nextInt();

        if (numero % 2 == 0) {
            System.out.println(numero + " es par");
        } else {
            System.out.println(numero + " es impar");
        }

        sc.close();
    }
}
```

2.2 ¿Son iguales?

Palabras clave: `#iguales` `#comparar`

Idea: para comparar se usa `==` (dos iguales). Un solo `=` sirve para guardar y aquí daría error.

Código completo · `Ej2_2_Iguales.java`

```java
import java.util.Scanner;

/*
 * 2.2 Pedir dos números enteros y decir si son iguales o no.
 *
 * Idea: se comparan con == (dos iguales), no con = (que sirve para guardar).
 */
public class Ej2_2_Iguales {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num1, num2;

        System.out.print("Primer número: ");
        num1 = sc.nextInt();
        System.out.print("Segundo número: ");
```

```
        num2 = sc.nextInt();

        if (num1 == num2) {
            System.out.println("Son iguales");
        } else {
            System.out.println("Son distintos");
        }

        sc.close();
    }
}
```

2.3 El mayor de dos

Palabras clave: `#mayor` `#dos-numeros`

Idea: el enunciado asegura que son distintos, así que solo hay dos posibilidades: un `if-else`.

Código completo · `Ej2_3_Mayor.java`

```
import java.util.Scanner;

/*
 * 2.3 Solicitar dos números distintos y mostrar cuál es el mayor.
 *
 * Idea: como el enunciado dice que son distintos, basta con un if-else.
 */
public class Ej2_3_Mayor {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num1, num2;

        System.out.print("Primer número: ");
        num1 = sc.nextInt();
        System.out.print("Segundo número (distinto del primero): ");
        num2 = sc.nextInt();

        if (num1 > num2) {
            System.out.println("El mayor es " + num1);
        } else {
            System.out.println("El mayor es " + num2);
        }

        sc.close();
    }
}
```

2.4 Casi-cero

Palabras clave: `#casi-cero` `#rango` `#decimal`

Qué pide: casi-cero = a menos de 1 del 0 (0.3, -0.99), pero el 0 no cuenta.

Idea: traducir la frase a condiciones unidas con `&&`: mayor que -1, menor que 1 y distinto de 0. El -1 y el 1 no valen, por eso es `>` y `<`. Se lee con `nextDouble()`.

Código completo · `Ej2_4_CasiCero.java`

```
import java.util.Scanner;

/*
 * 2.4 Pedir un número decimal e indicar si es "casi-cero": los que están a menos de
```

```
 * 1 unidad del 0 (positivos o negativos), pero sin contar el propio 0.
 * Ejemplos casi-cero: 0.3, -0.99, 0.123. No lo son: 12.3, 0, -1.
 *
 * Idea: tiene que estar entre -1 y 1 (sin incluirlos) Y además no ser 0.
 */
public class Ej2_4_CasiCero {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double numero;

        System.out.print("Introduce un número decimal: ");
        numero = sc.nextDouble();

        if (numero > -1 && numero < 1 && numero != 0) {
            System.out.println(numero + " es casi-cero");
        } else {
            System.out.println(numero + " no es casi-cero");
        }

        sc.close();
    }
}
```

2.5 Dos números de mayor a menor

Palabras clave: `#ordenar` `#decreciente` `#dos-numeros`

Idea: si el primero es mayor o igual, va delante; si no, va detrás. **Para cambiarlo:** de menor a mayor, `>=` pasa a `<=`.

Código completo · `Ej2_5_Decreciente.java`

```
import java.util.Scanner;

/*
 * 2.5 Pedir dos números y mostrarlos ordenados de forma decreciente (de mayor a menor).
 */
public class Ej2_5_Decreciente {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num1, num2;

        System.out.print("Primer número: ");
        num1 = sc.nextInt();
        System.out.print("Segundo número: ");
        num2 = sc.nextInt();

        // Si el primero es mayor (o igual) va delante; si no, va detrás
        if (num1 >= num2) {
            System.out.println(num1 + " " + num2);
        } else {
            System.out.println(num2 + " " + num1);
        }

        sc.close();
    }
}
```

2.6 El mayor, pudiendo ser iguales

Palabras clave: `#mayor` `#iguales` `#tres-casos`

Idea: ahora hay **tres** casos → `if - else if - else`. El último `else` es "iguales", porque si no es mayor ninguno solo pueden ser iguales.

Código completo · `Ej2_6_MayorOIguales.java`

```java
import java.util.Scanner;

/*
 * 2.6 Repetir el 2.3 (mostrar el mayor de dos números) teniendo en cuenta
 * que los números introducidos pueden ser iguales.
 *
 * Idea: ahora hay tres casos (mayor el primero, mayor el segundo o iguales),
 * así que se usa if - else if - else.
 */
public class Ej2_6_MayorOIguales {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num1, num2;

        System.out.print("Primer número: ");
        num1 = sc.nextInt();
        System.out.print("Segundo número: ");
        num2 = sc.nextInt();

        if (num1 > num2) {
            System.out.println("El mayor es " + num1);
        } else if (num2 > num1) {
            System.out.println("El mayor es " + num2);
        } else {
            System.out.println("Los dos números son iguales");
        }

        sc.close();
    }
}
```

2.7 Ordenar tres números

Palabras clave: `#ordenar` `#tres-numeros` `#intercambio`

Idea: probar las 6 combinaciones es largo y fácil de fallar. Es más sencillo **ir colocando** con intercambios, como ordenar tres cartas en la mano:

1. Si `a < b`, los cambio. Ahora a ≥ b.
2. Si `b < c`, los cambio. Ahora el más pequeño está en c.
3. Si `a < b`, los cambio otra vez. Ahora el más grande está en a.

Para intercambiar hace falta una variable `aux`, como cambiar el agua de dos vasos con un tercer vaso vacío. Ejemplo 3, 9, 5 → 9, 3, 5 → 9, 5, 3.

Código completo · `Ej2_7_OrdenarTres.java`

```java
import java.util.Scanner;

/*
 * 2.7 Pedir tres números y mostrarlos ordenados de mayor a menor.
 *
```

```java
 * Idea: en vez de probar las 6 combinaciones posibles, vamos "colocando" los números
 * con intercambios (como ordenar 3 cartas en la mano):
 *   1) si a < b, los cambio  -> ahora a >= b
 *   2) si b < c, los cambio  -> ahora c es el más pequeño
 *   3) si a < b, los cambio  -> ahora a es el más grande
 * Para intercambiar dos variables hace falta una variable auxiliar.
 */
public class Ej2_7_OrdenarTres {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a, b, c, aux;

        System.out.print("Primer número: ");
        a = sc.nextInt();
        System.out.print("Segundo número: ");
        b = sc.nextInt();
        System.out.print("Tercer número: ");
        c = sc.nextInt();

        if (a < b) {
            aux = a;
            a = b;
            b = aux;
        }
        if (b < c) {
            aux = b;
            b = c;
            c = aux;
        }
        if (a < b) {
            aux = a;
            a = b;
            b = aux;
        }

        System.out.println("De mayor a menor: " + a + " " + b + " " + c);

        sc.close();
    }
}
```

2.8 Ecuación de segundo grado

Palabras clave: `#ecuacion` `#segundo-grado` `#raiz` `#discriminante`

Idea: todo depende del **discriminante** b² – 4ac:

Discriminante	Soluciones
Mayor que 0	Dos: (–b + √d) / 2a y (–b – √d) / 2a
Igual a 0	Una: –b / 2a
Menor que 0	Ninguna real

Si `a == 0` no es de segundo grado (se dividiría entre 0) y se avisa. La raíz se hace con `Math.sqrt(...)`. Con a=1, b=-3, c=2 → x1 = 2, x2 = 1.

Código completo · `Ej2_8_SegundoGrado.java`

```java
import java.util.Scanner;
```

```java
/*
 * 2.8 Pedir los coeficientes de una ecuación de segundo grado (ax² + bx + c = 0)
 * y mostrar sus soluciones reales. Si no existen, indicarlo.
 *
 * Idea: todo depende del discriminante (b² - 4ac):
 *   - mayor que 0  -> dos soluciones
 *   - igual a 0    -> una solución
 *   - menor que 0  -> no hay soluciones reales
 * Si a vale 0 no es de segundo grado (habría que dividir entre 0), así que se avisa.
 */
public class Ej2_8_SegundoGrado {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double a, b, c, discriminante, x1, x2;

        System.out.print("Coeficiente a: ");
        a = sc.nextDouble();
        System.out.print("Coeficiente b: ");
        b = sc.nextDouble();
        System.out.print("Coeficiente c: ");
        c = sc.nextDouble();

        if (a == 0) {
            System.out.println("Si a es 0 no es una ecuación de segundo grado");
        } else {
            discriminante = b * b - 4 * a * c;

            if (discriminante > 0) {
                x1 = (-b + Math.sqrt(discriminante)) / (2 * a);
                x2 = (-b - Math.sqrt(discriminante)) / (2 * a);
                System.out.println("Soluciones: x1 = " + x1 + "  x2 = " + x2);
            } else if (discriminante == 0) {
                x1 = -b / (2 * a);
                System.out.println("Solución única: x = " + x1);
            } else {
                System.out.println("No tiene soluciones reales");
            }
        }

        sc.close();
    }
}
```

2.9 ¿Cuántas cifras tiene?

Palabras clave: `#cifras` `#cuantas-cifras` `#rango`

Idea: comparar con los límites de menor a mayor (< 10, < 100, < 1000...). Como `else if` va de arriba abajo, cuando llega a `< 100` ya sabe que no era menor que 10.

Código completo · `Ej2_9_Cifras.java`

```java
import java.util.Scanner;

/*
 * 2.9 Indicar cuántas cifras tiene un número entero entre 0 y 99.999.
 *
 * Idea: se compara con los límites: menos de 10 tiene 1 cifra, menos de 100 tiene 2...
 * Como el if-else if va de arriba abajo, en cada paso ya sabemos que no era de los
anteriores.
 */
public class Ej2_9_Cifras {
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numero, cifras;

        System.out.print("Introduce un número entre 0 y 99999: ");
        numero = sc.nextInt();

        if (numero < 0 || numero > 99999) {
            System.out.println("Número fuera de rango");
        } else {
            if (numero < 10) {
                cifras = 1;
            } else if (numero < 100) {
                cifras = 2;
            } else if (numero < 1000) {
                cifras = 3;
            } else if (numero < 10000) {
                cifras = 4;
            } else {
                cifras = 5;
            }
            System.out.println(numero + " tiene " + cifras + " cifras");
        }

        sc.close();
    }
}
```

2.10 Nota en texto

Palabras clave: `#nota` `#texto` `#switch` `#tramos`

Idea: son valores concretos → `switch`. Varios `case` seguidos sin `break` comparten resultado. `default` recoge lo que no está (11, -2...).

Código completo · `Ej2_10_NotaTexto.java`

```
import java.util.Scanner;

/*
 * 2.10 Pedir una nota entera de 0 a 10 y mostrarla así: insuficiente (0 a 4),
 * suficiente (5), bien (6), notable (7 y 8) y sobresaliente (9 y 10).
 *
 * Idea: son valores concretos, así que un switch queda muy claro.
 * Varios case seguidos sin break comparten el mismo resultado.
 */
public class Ej2_10_NotaTexto {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int nota;

        System.out.print("Introduce la nota (0-10): ");
        nota = sc.nextInt();

        switch (nota) {
            case 0:
            case 1:
            case 2:
            case 3:
            case 4:
                System.out.println("Insuficiente");
```

```
                break;
            case 5:
                System.out.println("Suficiente");
                break;
            case 6:
                System.out.println("Bien");
                break;
            case 7:
            case 8:
                System.out.println("Notable");
                break;
            case 9:
            case 10:
                System.out.println("Sobresaliente");
                break;
            default:
                System.out.println("Nota no válida");
        }

        sc.close();
    }
}
```

2.11 Día de la semana

Palabras clave: `#dia-semana` `#switch` `#texto`

Idea: mismo esquema: un `case` por día y `default` para números no válidos.

Código completo · `Ej2_11_DiaSemana.java`

```
import java.util.Scanner;

/*
 * 2.11 Pedir un número del 1 al 7 y mostrar el día de la semana (1 = lunes ... 7 = domingo).
 */
public class Ej2_11_DiaSemana {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int dia;

        System.out.print("Introduce un número del 1 al 7: ");
        dia = sc.nextInt();

        switch (dia) {
            case 1:
                System.out.println("Lunes");
                break;
            case 2:
                System.out.println("Martes");
                break;
            case 3:
                System.out.println("Miércoles");
                break;
            case 4:
                System.out.println("Jueves");
                break;
            case 5:
                System.out.println("Viernes");
                break;
            case 6:
                System.out.println("Sábado");
```

```
                break;
            case 7:
                System.out.println("Domingo");
                break;
            default:
                System.out.println("Número no válido");
        }

        sc.close();
    }
}
```

2.12 ¿Fecha correcta?

Palabras clave: `#fecha` `#fecha-valida` `#dias-mes` `#switch`

Idea: dos pasos más fáciles:

1. Averiguar **cuántos días tiene el mes** con un `switch` (febrero 28; abril, junio, septiembre, noviembre 30; el resto 31; un mes que no existe, 0).
2. Correcta si `dia >= 1 && dia <= diasDelMes`.

Con el truco del 0, un mes 13 no admite ningún día y no hace falta comprobarlo aparte.

Código completo · `Ej2_12_FechaCorrecta.java`

```
import java.util.Scanner;

/*
 * 2.12 Pedir día, mes y año e indicar si la fecha es correcta.
 * Hay meses de 28, 30 y 31 días (sin tener en cuenta años bisiestos).
 *
 * Idea: primero se calcula cuántos días tiene ese mes (con un switch)
 * y luego se comprueba que el día esté entre 1 y ese número.
 */
public class Ej2_12_FechaCorrecta {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int dia, mes, anio, diasDelMes;

        System.out.print("Día: ");
        dia = sc.nextInt();
        System.out.print("Mes: ");
        mes = sc.nextInt();
        System.out.print("Año: ");
        anio = sc.nextInt();

        switch (mes) {
            case 2:
                diasDelMes = 28;
                break;
            case 4:
            case 6:
            case 9:
            case 11:
                diasDelMes = 30;
                break;
            case 1:
            case 3:
            case 5:
            case 7:
            case 8:
```

```
            case 10:
            case 12:
                diasDelMes = 31;
                break;
            default:
                diasDelMes = 0; // mes que no existe: ningún día será válido
        }

        if (dia >= 1 && dia <= diasDelMes) {
            System.out.println(dia + "/" + mes + "/" + anio + " es una fecha correcta");
        } else {
            System.out.println(dia + "/" + mes + "/" + anio + " no es una fecha correcta");
        }

        sc.close();
    }
}
```

2.13 Un segundo más tarde

Palabras clave: `#hora` `#segundo-siguiente` `#reloj` `#ceros-delante`

Idea: como el cuentakilómetros de un coche: sumo 1 al segundo; si llega a 60 vuelve a 0 y sumo 1 al minuto; si el minuto llega a 60, igual con la hora; si la hora llega a 24, vuelve a 0. Para que salga `10:42:00` y no `10:42:0`, el ternario pone un 0 delante: `(segundo < 10 ? "0" : "") + segundo`.

Código completo · `Ej2_13_SegundoMas.java`

```
import java.util.Scanner;

/*
 * 2.13 Pedir una hora (horas, minutos y segundos) y mostrar qué hora será un segundo más tarde.
 * Ejemplo: 10:41:59 -> 10:42:00
 *
 * Idea: sumo 1 al segundo. Si llega a 60 vuelve a 0 y sumo 1 al minuto.
 * Si el minuto llega a 60, vuelve a 0 y sumo 1 a la hora. Si la hora llega a 24, vuelve a 0.
 * (Es como el cuentakilómetros de un coche: cuando una rueda da la vuelta, mueve la siguiente.)
 */
public class Ej2_13_SegundoMas {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int hora, minuto, segundo;

        System.out.print("Hora: ");
        hora = sc.nextInt();
        System.out.print("Minutos: ");
        minuto = sc.nextInt();
        System.out.print("Segundos: ");
        segundo = sc.nextInt();

        if (hora < 0 || hora > 23 || minuto < 0 || minuto > 59 || segundo < 0 || segundo > 59) {
            System.out.println("Hora no válida");
        } else {
            segundo++;
            if (segundo == 60) {
                segundo = 0;
                minuto++;
```

```
                if (minuto == 60) {
                    minuto = 0;
                    hora++;
                    if (hora == 24) {
                        hora = 0;
                    }
                }
            }
            // El ternario pone un 0 delante si el número tiene una cifra (9 -> "09")
            System.out.println("Un segundo después: "
                    + (hora < 10 ? "0" : "") + hora + ":"
                    + (minuto < 10 ? "0" : "") + minuto + ":"
                    + (segundo < 10 ? "0" : "") + segundo);
        }

        sc.close();
    }
}
```

2.14 Día siguiente

Palabras clave: `#fecha` `#dia-siguiente` `#dias-mes`

Idea: el 2.12 y el 2.13 juntos: calculo los días del mes, sumo 1 al día; si se pasa, día = 1 y mes + 1; si el mes pasa de 12, mes = 1 y año + 1. 28/2/2025 → 1/3/2025; 31/12/2025 → 1/1/2026.

Código completo · `Ej2_14_DiaSiguiente.java`

```
import java.util.Scanner;

/*
 * 2.14 Pedir una fecha (día, mes y año) y mostrar la fecha del día siguiente.
 * (Sin años bisiestos, igual que en el 2.12.)
 *
 * Idea: es la unión del 2.12 y el 2.13. Calculo los días del mes, sumo 1 al día;
 * si se pasa de los días del mes, día = 1 y paso al mes siguiente; si el mes pasa de 12,
 * mes = 1 y paso al año siguiente.
 */
public class Ej2_14_DiaSiguiente {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int dia, mes, anio, diasDelMes;

        System.out.print("Día: ");
        dia = sc.nextInt();
        System.out.print("Mes: ");
        mes = sc.nextInt();
        System.out.print("Año: ");
        anio = sc.nextInt();

        switch (mes) {
            case 2:
                diasDelMes = 28;
                break;
            case 4:
            case 6:
            case 9:
            case 11:
                diasDelMes = 30;
                break;
            case 1:
            case 3:
```

```
            case 5:
            case 7:
            case 8:
            case 10:
            case 12:
                diasDelMes = 31;
                break;
            default:
                diasDelMes = 0;
        }

        if (dia < 1 || dia > diasDelMes) {
            System.out.println("La fecha no es correcta");
        } else {
            dia++;
            if (dia > diasDelMes) {
                dia = 1;
                mes++;
                if (mes > 12) {
                    mes = 1;
                    anio++;
                }
            }
            System.out.println("El día siguiente es: " + dia + "/" + mes + "/" + anio);
        }

        sc.close();
    }
}
```

UD2 P2. Condicionales y aleatorios (2.11 a 2.19)

Herramientas nuevas: **sacar cifras** con `%` y `/`, **números aleatorios** y el **operador ternario**.

2.11 Capicúa

Palabras clave: `#capicua` `#cifras` `#al-reves`

Idea: darle la vuelta al número y comparar con el original. `numero % 10` da la última cifra y `numero / 10` la quita. Se construye al revés: `alReves = alReves * 10 + cifra`.

Vuelta	copia	cifra	alReves
1	1221	1	1
2	122	2	12
3	12	2	122
4	1	1	1221

Sirve para cualquier cantidad de cifras. Se trabaja con una copia para no perder el original.

Código completo · `Ej2_11_Capicua.java`

```
import java.util.Scanner;

/*
 * 2.11 Pedir un número entre 0 y 9.999 e indicar si es capicúa
 * (se lee igual de izquierda a derecha que de derecha a izquierda: 1221, 44, 7).
 *
 * Idea: le doy la vuelta al número cifra a cifra y comparo con el original.
```

```
 *   - numero % 10  -> me da la última cifra
 *   - numero / 10  -> le quita la última cifra
 * Voy construyendo el número al revés: alReves = alReves * 10 + cifra.
 * Así sirve para cualquier cantidad de cifras.
 */
public class Ej2_11_Capicua {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numero, copia, alReves = 0;

        System.out.print("Introduce un número entre 0 y 9999: ");
        numero = sc.nextInt();

        if (numero < 0 || numero > 9999) {
            System.out.println("Número fuera de rango");
        } else {
            copia = numero; // trabajo con una copia para no perder el original
            while (copia > 0) {
                alReves = alReves * 10 + copia % 10;
                copia = copia / 10;
            }

            if (alReves == numero) {
                System.out.println(numero + " es capicúa");
            } else {
                System.out.println(numero + " no es capicúa");
            }
        }

        sc.close();
    }
}
```

2.12 Letra del DNI

Palabras clave: `#dni` `#letra` `#charAt` `#resto`

Idea: el resto de dividir entre 23 dice la letra. **Ojo:** el enunciado dice "módulo 22", pero la tabla tiene 23 letras (0 a 22); con 22 la E no saldría nunca. **Truco:** las letras en orden en un texto y `charAt(resto)` en vez de un `switch` de 23 casos (si no habéis visto `charAt`, el `switch` también vale). 12345678 → Z.

Código completo · `Ej2_12_LetraDNI.java`

```
import java.util.Scanner;

/*
 * 2.12 Calcular la letra del DNI a partir de su número.
 * Tabla: 0=T 1=R 2=W 3=A 4=G 5=M 6=Y 7=F 8=P 9=D 10=X 11=B 12=N 13=J 14=Z 15=S
 *        16=Q 17=V 18=H 19=L 20=C 21=K 22=E
 *
 * Idea: el resto de dividir el número entre 23 dice la posición de la letra.
 * OJO: el enunciado pone "módulo 22", pero la tabla tiene 23 letras (de 0 a 22),
 * así que el resto tiene que ser entre 23 (si fuera 22, la E no saldría nunca).
 *
 * Truco: en vez de un switch de 23 casos, guardo las letras en orden en un String
 * y saco la que está en la posición del resto con charAt.
 */
public class Ej2_12_LetraDNI {

    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
        final String LETRAS = "TRWAGMYFPDXBNJZSQVHLCKE";
        int dni, resto;
        char letra;

        System.out.print("Introduce el número del DNI (8 cifras): ");
        dni = sc.nextInt();

        if (dni < 0 || dni > 99999999) {
            System.out.println("Número de DNI no válido");
        } else {
            resto = dni % 23;
            letra = LETRAS.charAt(resto);
            System.out.println("DNI completo: " + dni + letra);
        }

        sc.close();
    }
}
```

2.13 Comida de la granja

Palabras clave: `#dividir-cero` `#granja` `#repartir`

Pasos: 1) si no hay animales, avisar (así nunca se divide entre 0); 2) comida necesaria = animales · kilos por animal; 3) si la comprada llega, bien; si no, ración = comida / animales. 10 kg, 4 animales de 3 kg → no llega, 2.5 kg cada uno.

Código completo · `Ej2_13_Granja.java`

```
import java.util.Scanner;

/*
 * 2.13 En una granja se compran comidaDiaria kilos de comida al día para numAnimales
 * animales, y cada uno come de media kilosPorAnimal. Decir si hay comida suficiente.
 * Si no la hay, calcular la ración que le toca a cada animal.
 * Nota: evitar divisiones entre cero.
 *
 * Idea: comida necesaria = numAnimales * kilosPorAnimal.
 * Si la comprada llega, perfecto. Si no, se reparte: comidaDiaria / numAnimales.
 * Antes de dividir compruebo que numAnimales no sea 0.
 */
public class Ej2_13_Granja {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double comidaDiaria, kilosPorAnimal, necesaria, racion;
        int numAnimales;

        System.out.print("Kilos de comida comprados al día: ");
        comidaDiaria = sc.nextDouble();
        System.out.print("Número de animales: ");
        numAnimales = sc.nextInt();
        System.out.print("Kilos que come cada animal de media: ");
        kilosPorAnimal = sc.nextDouble();

        if (numAnimales <= 0) {
            System.out.println("No hay animales que alimentar");
        } else {
            necesaria = numAnimales * kilosPorAnimal;
            if (comidaDiaria >= necesaria) {
                System.out.println("Hay comida suficiente para todos");
            } else {
```

```
                racion = comidaDiaria / numAnimales;
                System.out.println("No hay comida suficiente. A cada animal le tocan " +
racion + " kg");
            }
        }

        sc.close();
    }
}
```

2.14 Número en letras

Palabras clave: `#texto` `#numero-en-letras` `#switch` `#cifras`

Idea: decenas = `numero / 10`, unidades = `numero % 10`. Tres zonas en castellano:

Zona	Regla	Ejemplo
1 a 19	Nombre propio	once, quince
20 a 29	Todo junto: "veinti" + unidad	veintiuno, veintidós
30 a 99	Decena + " y " + unidad (solo decena si acaba en 0)	treinta, cuarenta y cuatro

Veintidós, veintitrés y veintiséis llevan tilde y tienen su propio `case`.

Código completo · `Ej2_14_NumeroEnLetras.java`

```java
import java.util.Scanner;

/*
 * 2.14 Pedir un número entre 1 y 99 y mostrarlo con letras (56 -> "cincuenta y seis").
 *
 * Idea: separo decenas (numero / 10) y unidades (numero % 10).
 * Hay tres zonas con reglas distintas en castellano:
 *   - 1 a 19: tienen nombre propio (once, doce, quince...)
 *   - 20 a 29: se escriben juntos (veintiuno, veintidós...)
 *   - 30 a 99: decena + " y " + unidad (treinta y cuatro), o solo la decena si acaba en 0
 */
public class Ej2_14_NumeroEnLetras {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numero, decenas, unidades;
        String textoUnidad = "", textoDecena = "", resultado = "";

        System.out.print("Introduce un número entre 1 y 99: ");
        numero = sc.nextInt();

        if (numero < 1 || numero > 99) {
            System.out.println("Número fuera de rango");
        } else {
            decenas = numero / 10;
            unidades = numero % 10;

            switch (unidades) {
                case 1: textoUnidad = "uno"; break;
                case 2: textoUnidad = "dos"; break;
                case 3: textoUnidad = "tres"; break;
                case 4: textoUnidad = "cuatro"; break;
                case 5: textoUnidad = "cinco"; break;
                case 6: textoUnidad = "seis"; break;
                case 7: textoUnidad = "siete"; break;
                case 8: textoUnidad = "ocho"; break;
```

```
                case 9: textoUnidad = "nueve"; break;
            }

            if (numero < 10) {
                resultado = textoUnidad;
            } else if (numero < 20) {
                switch (numero) {
                    case 10: resultado = "diez"; break;
                    case 11: resultado = "once"; break;
                    case 12: resultado = "doce"; break;
                    case 13: resultado = "trece"; break;
                    case 14: resultado = "catorce"; break;
                    case 15: resultado = "quince"; break;
                    case 16: resultado = "dieciséis"; break;
                    case 17: resultado = "diecisiete"; break;
                    case 18: resultado = "dieciocho"; break;
                    case 19: resultado = "diecinueve"; break;
                }
            } else if (numero < 30) {
                switch (unidades) {
                    case 0: resultado = "veinte"; break;
                    case 2: resultado = "veintidós"; break;   // llevan tilde
                    case 3: resultado = "veintitrés"; break;
                    case 6: resultado = "veintiséis"; break;
                    default: resultado = "veinti" + textoUnidad;
                }
            } else {
                switch (decenas) {
                    case 3: textoDecena = "treinta"; break;
                    case 4: textoDecena = "cuarenta"; break;
                    case 5: textoDecena = "cincuenta"; break;
                    case 6: textoDecena = "sesenta"; break;
                    case 7: textoDecena = "setenta"; break;
                    case 8: textoDecena = "ochenta"; break;
                    case 9: textoDecena = "noventa"; break;
                }
                if (unidades == 0) {
                    resultado = textoDecena;
                } else {
                    resultado = textoDecena + " y " + textoUnidad;
                }
            }
            System.out.println(resultado);
        }

        sc.close();
    }
}
```

2.15 Área del triángulo

Palabras clave: `#area` `#triangulo` `#negativos`

Idea: área = base · altura / 2, comprobando antes que ninguno sea negativo (`base < 0 || altura < 0`).

Código completo · `Ej2_15_AreaTriangulo.java`

```
import java.util.Scanner;

/*
 * 2.15 Pedir la base y la altura de un triángulo (números reales) y mostrar su área,
 * comprobando que no son negativos.
```

```
 *
 * Fórmula: área = base * altura / 2
 */
public class Ej2_15_AreaTriangulo {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double base, altura, area;

        System.out.print("Base: ");
        base = sc.nextDouble();
        System.out.print("Altura: ");
        altura = sc.nextDouble();

        if (base < 0 || altura < 0) {
            System.out.println("La base y la altura no pueden ser negativas");
        } else {
            area = base * altura / 2;
            System.out.println("El área es " + area);
        }

        sc.close();
    }
}
```

2.16 Valor absoluto con ternario

Palabras clave: `#ternario` `#valor-absoluto`

Idea: si es negativo se le cambia el signo; si no, se queda igual: `absoluto = numero < 0 ? -numero : numero;`

Código completo · `Ej2_16_ValorAbsoluto.java`

```
import java.util.Scanner;

/*
 * 2.16 Usar el operador ternario para calcular el valor absoluto de un número.
 * (El valor absoluto es el número sin signo: |-5| = 5, |3| = 3.)
 *
 * Ternario:  variable = condicion ? valorSiTrue : valorSiFalse;
 * Si el número es negativo le cambio el signo; si no, se queda igual.
 */
public class Ej2_16_ValorAbsoluto {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double numero, absoluto;

        System.out.print("Introduce un número: ");
        numero = sc.nextDouble();

        absoluto = numero < 0 ? -numero : numero;

        System.out.println("El valor absoluto de " + numero + " es " + absoluto);

        sc.close();
    }
}
```

2.17 Juego de la suma

Palabras clave: `#aleatorio` `#juego` `#suma` `#random`

El aleatorio `(int) (Math.random() * 99) + 1`: `Math.random()` da 0 a 0.999...; `* 99` → 0 a 98.99; `(int)` quita decimales → 0 a 98; `+ 1` → 1 a 99. **Regla:** entre 1 y N → `(int) (Math.random() * N) + 1`.

Código completo · `Ej2_17_JuegoSuma.java`

```java
import java.util.Scanner;

/*
 * 2.17 "Juego de la suma": salen dos números aleatorios entre 1 y 99, el usuario
 * escribe la suma y el programa dice si es correcta.
 *
 * Aleatorio entre 1 y N:  (int) (Math.random() * N) + 1
 *   Math.random() da un decimal entre 0 y 0.999..., al multiplicar por 99 va de 0 a 98.99,
 *   (int) quita los decimales (0 a 98) y el +1 lo deja entre 1 y 99.
 */
public class Ej2_17_JuegoSuma {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num1, num2, respuesta, resultado;

        num1 = (int) (Math.random() * 99) + 1;
        num2 = (int) (Math.random() * 99) + 1;
        resultado = num1 + num2;

        System.out.print("¿Cuánto es " + num1 + " + " + num2 + "? ");
        respuesta = sc.nextInt();

        if (respuesta == resultado) {
            System.out.println("¡Correcto!");
        } else {
            System.out.println("Incorrecto, era " + resultado);
        }

        sc.close();
    }
}
```

2.18 Juego con operación aleatoria

Palabras clave: `#aleatorio` `#juego` `#operaciones` `#switch`

Idea: otro aleatorio de 0 a 2 elige la operación y un `switch` calcula el resultado y el símbolo. **Para cambiarlo:** con división sería `* 4` y un `case 3`.

Código completo · `Ej2_18_JuegoOperaciones.java`

```java
import java.util.Scanner;

/*
 * 2.18 Como el 2.17, pero la operación también es aleatoria: suma, resta o multiplicación.
 *
 * Idea: saco otro aleatorio de 0 a 2 que decide la operación (0 suma, 1 resta, 2
multiplicación)
 * y con un switch calculo el resultado y elijo el símbolo que se muestra.
 */
public class Ej2_18_JuegoOperaciones {
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num1, num2, operacion, respuesta;
        int resultado = 0;
        char simbolo = ' ';

        num1 = (int) (Math.random() * 99) + 1;
        num2 = (int) (Math.random() * 99) + 1;
        operacion = (int) (Math.random() * 3); // 0, 1 o 2

        switch (operacion) {
            case 0:
                simbolo = '+';
                resultado = num1 + num2;
                break;
            case 1:
                simbolo = '-';
                resultado = num1 - num2;
                break;
            case 2:
                simbolo = '*';
                resultado = num1 * num2;
                break;
        }

        System.out.print("¿Cuánto es " + num1 + " " + simbolo + " " + num2 + "? ");
        respuesta = sc.nextInt();

        if (respuesta == resultado) {
            System.out.println("¡Correcto!");
        } else {
            System.out.println("Incorrecto, era " + resultado);
        }

        sc.close();
    }
}
```

2.19 Grados a radianes

Palabras clave: `#conversion` `#grados` `#radianes` `#modulo`

Pasos: 1) `grados % 360` quita vueltas completas (370 → 10); 2) si era negativo se suma 360 (-90 → 270); 3) radianes = grados · `Math.PI` / 180.

Código completo · `Ej2_19_GradosRadianes.java`

```
import java.util.Scanner;

/*
 * 2.19 Pedir los grados de un ángulo y mostrar su equivalente en radianes.
 * Si no está entre 0 y 360, pasarlo a ese rango (el módulo ayuda).
 *
 * Idea:
 *   - grados % 360 quita las vueltas completas (370 -> 10, 720 -> 0).
 *   - Si era negativo, el % sigue dando negativo (-90 % 360 = -90), así que le sumo 360 (->
270).
 *   - Radianes = grados * PI / 180
 */
public class Ej2_19_GradosRadianes {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        double grados, radianes;

        System.out.print("Grados del ángulo: ");
        grados = sc.nextDouble();

        grados = grados % 360;
        if (grados < 0) {
            grados = grados + 360;
        }

        radianes = grados * Math.PI / 180;

        System.out.println("Ángulo en el rango 0-360: " + grados + " grados");
        System.out.println("En radianes: " + radianes);

        sc.close();
    }
}
```

Tema 2. Ejercicios adicionales (24 a 26)

24. Edades y alturas de 5 alumnos

Palabras clave: `#media` `#contador` `#acumulador` `#alumnos`

Idea: un `for` de 5 vueltas con dos acumuladores (`sumaEdades`, `sumaAlturas`) y dos contadores (`mayores18`, `altos`). Los contadores van con dos `if` **separados**, porque un alumno puede ser mayor y alto a la vez. Las medias se calculan fuera del bucle. Sin bucles (tema 2) habría que repetir el bloque 5 veces.

Código completo · `Ej24_Alumnos.java`

```
import java.util.Scanner;

/*
 * Ejercicio 24. Dadas las edades y alturas de 5 alumnos, mostrar la edad y la estatura
media,
 * la cantidad de alumnos mayores de 18 años y la cantidad que miden más de 1.75.
 *
 * Idea: un for de 5 vueltas. En cada vuelta pido edad y altura y:
 *   - las sumo a sus acumuladores (para la media)
 *   - si cumple la condición, sumo 1 al contador
 * Las medias se calculan al final, fuera del bucle.
 * (Si aún no habéis visto bucles, se puede repetir el bloque 5 veces, pero queda mucho más
largo.)
 */
public class Ej24_Alumnos {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        final int NUM_ALUMNOS = 5;
        int edad, sumaEdades = 0, mayores18 = 0, altos = 0;
        double altura, sumaAlturas = 0;

        for (int alumno = 1; alumno <= NUM_ALUMNOS; alumno++) {
            System.out.print("Edad del alumno " + alumno + ": ");
            edad = sc.nextInt();
            System.out.print("Altura del alumno " + alumno + " (m): ");
            altura = sc.nextDouble();

            sumaEdades = sumaEdades + edad;
```

```
            sumaAlturas = sumaAlturas + altura;

            if (edad > 18) {
                mayores18++;
            }
            if (altura > 1.75) {
                altos++;
            }
        }

        System.out.println("Edad media: " + (double) sumaEdades / NUM_ALUMNOS);
        System.out.println("Altura media: " + sumaAlturas / NUM_ALUMNOS);
        System.out.println("Mayores de 18: " + mayores18);
        System.out.println("Miden más de 1.75: " + altos);

        sc.close();
    }
}
```

25. Millas a km

Palabras clave: `#conversion` `#millas` `#km` `#constante`

Idea: km = millas · 1.6. El 1.6 va en una constante `KM_POR_MILLA`: se entiende qué es y se cambia en un solo sitio.

Código completo · `Ej25_MillasKm.java`

```
import java.util.Scanner;

/*
 * Ejercicio 25. Leer millas (double) y convertirlas a km. 1 milla = 1.6 km.
 *
 * Buena práctica: el 1.6 se guarda en una constante (final) con nombre en mayúsculas,
 * así se entiende qué es y, si cambia, solo se toca en un sitio.
 */
public class Ej25_MillasKm {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        final double KM_POR_MILLA = 1.6;
        double millas, km;

        System.out.print("Millas: ");
        millas = sc.nextDouble();

        km = millas * KM_POR_MILLA;

        System.out.println(millas + " millas son " + km + " km");

        sc.close();
    }
}
```

26. MostrarTiempo

Palabras clave: `#hora` `#minutos-segundos` `#MostrarTiempo`

Idea: `total / 60` = minutos completos; `total % 60` = segundos que sobran. 130 s → 2 min 10 s. **Para cambiarlo** con horas: `horas = total / 3600; minutos = total % 3600 / 60;`.

Código completo · `MostrarTiempo.java`

```java
import java.util.Scanner;

/*
 * Ejercicio 26. Programa MostrarTiempo.java: pedir una cantidad de segundos (entero positivo)
 * y mostrar cuántos minutos y segundos son.
 *
 * Idea: 1 minuto = 60 segundos.
 *   - minutos = total / 60  (división entera: cuántos 60 caben)
 *   - segundos = total % 60 (lo que sobra)
 * Ejemplo: 130 s -> 130 / 60 = 2 minutos y 130 % 60 = 10 segundos.
 */
public class MostrarTiempo {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int total, minutos, segundos;

        System.out.print("Cantidad de segundos: ");
        total = sc.nextInt();

        if (total < 0) {
            System.out.println("Tiene que ser un número positivo");
        } else {
            minutos = total / 60;
            segundos = total % 60;
            System.out.println(total + " segundos son " + minutos + " minutos y " + segundos + " segundos");
        }

        sc.close();
    }
}
```

UD3 I. Bucles básicos (3.1 a 3.15)

Si el enunciado dice…	Bucle
"hasta que meta 0 / un negativo / -1"	`while` con lectura antes y al final (centinela)
"asegurándose de que está en el rango" o "al menos una vez"	`do-while`
"pide 10 números", "del 1 al n", "las tablas del 1 al 10"	`for`

3.1 Par, positivo y cuadrado hasta el 0

Palabras clave: `#centinela` `#par` `#positivo` `#cuadrado`

Idea (centinela): el 0 es la señal de parar y no se procesa. Se lee **antes** del bucle y otra vez **al final** de cada vuelta. Si solo se leyera al principio de dentro, el 0 también se procesaría.

Código completo · `Ej3_1_ParPositivoCuadrado.java`

```java
import java.util.Scanner;

/*
 * 3.1 Para cada número introducido, mostrar si es par, si es positivo y su cuadrado.
 * Se repite hasta que el número introducido sea 0.
 *
```

```java
 * Idea (patrón "centinela"): leo un número ANTES del while, el while se repite mientras
 * no sea 0, y al final de cada vuelta vuelvo a leer. Así el 0 nunca se procesa.
 */
public class Ej3_1_ParPositivoCuadrado {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numero;

        System.out.print("Número (0 para salir): ");
        numero = sc.nextInt();

        while (numero != 0) {
            if (numero % 2 == 0) {
                System.out.println(numero + " es par");
            } else {
                System.out.println(numero + " es impar");
            }

            if (numero > 0) {
                System.out.println(numero + " es positivo");
            } else {
                System.out.println(numero + " es negativo");
            }

            System.out.println("Su cuadrado es " + numero * numero);

            System.out.print("Número (0 para salir): ");
            numero = sc.nextInt();
        }

        System.out.println("Fin");
        sc.close();
    }
}
```

3.2 Estadística de edades

Palabras clave: `#centinela` `#media` `#contador` `#acumulador` `#edades`

Idea: centinela (negativo) + acumulador (`suma`) + contadores (`alumnos`, `mayores`). La media se calcula **fuera** y solo si hay alumnos (para no dividir entre 0).

Código completo · `Ej3_2_EstadisticaEdades.java`

```java
import java.util.Scanner;

/*
 * 3.2 Pedir edades de alumnos hasta que se introduzca una negativa y mostrar:
 * la suma de las edades, la media, el número de alumnos y cuántos son mayores de edad.
 *
 * Idea: centinela (negativo) + acumulador (suma) + contadores (alumnos y mayores).
 * La media se calcula al final y solo si hay algún alumno (para no dividir entre 0).
 */
public class Ej3_2_EstadisticaEdades {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int edad, suma = 0, alumnos = 0, mayores = 0;

        System.out.print("Edad (negativo para terminar): ");
        edad = sc.nextInt();
```

```
        while (edad >= 0) {
            suma = suma + edad;
            alumnos++;
            if (edad >= 18) {
                mayores++;
            }
            System.out.print("Edad (negativo para terminar): ");
            edad = sc.nextInt();
        }

        if (alumnos == 0) {
            System.out.println("No se ha introducido ninguna edad");
        } else {
            System.out.println("Suma: " + suma);
            System.out.println("Media: " + (double) suma / alumnos);
            System.out.println("Alumnos: " + alumnos);
            System.out.println("Mayores de edad: " + mayores);
        }

        sc.close();
    }
}
```

3.3 Número secreto

Palabras clave: `#aleatorio` `#juego` `#adivinar` `#numero-secreto`

Idea: el bucle sigue mientras **no** acierte **y no** se rinda: `while (intento != secreto && intento != -1)`. Al salir, un `if` mira cuál de las dos pasó. **Para cambiarlo:** contar intentos con `intentos++`.

Código completo · `Ej3_3_NumeroSecreto.java`

```
import java.util.Scanner;

/*
 * 3.3 Juego del número secreto: adivinar un número aleatorio entre 1 y 100.
 * Por cada intento se dice "mayor" o "menor". Termina al acertar o al rendirse con -1.
 *
 * Idea: el while sigue mientras NO haya acertado Y NO se haya rendido.
 * Al salir, miro cuál de las dos cosas pasó para dar el mensaje.
 */
public class Ej3_3_NumeroSecreto {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int secreto, intento;

        secreto = (int) (Math.random() * 100) + 1;

        System.out.print("Adivina el número (1-100, -1 para rendirte): ");
        intento = sc.nextInt();

        while (intento != secreto && intento != -1) {
            if (secreto > intento) {
                System.out.println("Mayor");
            } else {
                System.out.println("Menor");
            }
            System.out.print("Otro intento: ");
            intento = sc.nextInt();
        }
```

```
        if (intento == secreto) {
            System.out.println("¡Acertaste!");
        } else {
            System.out.println("Te has rendido. Era el " + secreto);
        }

        sc.close();
    }
}
```

3.4 Árbol más alto

Palabras clave: `#maximo` `#centinela` `#mas-alto`

Idea (máximo): guardar el más alto visto hasta ahora (altura y número). Cada árbol nuevo se compara; si es más alto, pasa a ser el máximo. `alturaMax` empieza en -1 para que el primero siempre lo supere.

Código completo · `Ej3_4_ArbolMasAlto.java`

```
import java.util.Scanner;

/*
 * 3.4 Pedir la altura (cm) de cada árbol hasta introducir -1 y mostrar cuál es el más alto.
 * Los árboles se numeran 0, 1, 2... según se van introduciendo.
 *
 * Idea (patrón "máximo"): guardo la altura máxima vista hasta ahora y el número del árbol.
 * Si llega uno más alto, actualizo las dos cosas. Empiezo el máximo en -1 para que
 * el primer árbol siempre lo supere.
 */
public class Ej3_4_ArbolMasAlto {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int altura, arbol = 0;
        int alturaMax = -1, arbolMax = -1;

        System.out.print("Altura del árbol " + arbol + " (-1 para terminar): ");
        altura = sc.nextInt();

        while (altura != -1) {
            if (altura > alturaMax) {
                alturaMax = altura;
                arbolMax = arbol;
            }
            arbol++;
            System.out.print("Altura del árbol " + arbol + " (-1 para terminar): ");
            altura = sc.nextInt();
        }

        if (arbolMax == -1) {
            System.out.println("No se ha introducido ningún árbol");
        } else {
            System.out.println("El más alto es el árbol " + arbolMax + " con " + alturaMax +
" cm");
        }

        sc.close();
    }
}
```

3.5 Cálculo mental

Palabras clave: `#aleatorio` `#juego` `#do-while` `#aciertos`

Idea: `do-while` (al menos una suma) con un `boolean acierto` como condición.

Código completo · `Ej3_5_CalculoMental.java`

```java
import java.util.Scanner;

/*
 * 3.5 Juego de cálculo mental: sumas de dos aleatorios entre 1 y 100.
 * Mientras acierte, sigue. Al fallar, termina y muestra cuántas acertó.
 *
 * Idea: do-while porque al menos se juega una vez. La condición es "ha acertado".
 */
public class Ej3_5_CalculoMental {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int num1, num2, respuesta, aciertos = 0;
        boolean acierto;

        do {
            num1 = (int) (Math.random() * 100) + 1;
            num2 = (int) (Math.random() * 100) + 1;

            System.out.print(num1 + " + " + num2 + " = ");
            respuesta = sc.nextInt();

            acierto = respuesta == num1 + num2;
            if (acierto) {
                aciertos++;
            }
        } while (acierto);

        System.out.println("Has fallado. Operaciones correctas: " + aciertos);
        sc.close();
    }
}
```

3.6 Contar del 1 al n

Palabras clave: `#for` `#contar` `#del-1-al-n`

Idea: se sabe cuántas vueltas son → `for (int i = 1; i <= n; i++)`.

Código completo · `Ej3_6_Contar.java`

```java
import java.util.Scanner;

/*
 * 3.6 Pedir un número n y mostrar todos los números del 1 al n.
 *
 * Idea: sé cuántas vueltas son (n), así que uso un for de 1 a n.
 */
public class Ej3_6_Contar {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n;

        System.out.print("Introduce n: ");
        n = sc.nextInt();
```

```
        for (int i = 1; i <= n; i++) {
            System.out.println(i);
        }

        sc.close();
    }
}
```

3.7 Múltiplos de 7 menores que 100

Palabras clave: `#multiplos` `#for`

Idea eficiente: empezar en 7 y sumar 7 cada vez (14 vueltas en vez de 99). **Otra forma válida:** recorrer del 1 al 99 y escribir si `numero % 7 == 0`.

Código completo · `Ej3_7_Multiplos7.java`

```
/*
 * 3.7 Escribir todos los múltiplos de 7 menores que 100.
 *
 * Idea: empiezo en 7 y voy sumando 7 de cada vez. Así solo paso por los múltiplos
 * (14 vueltas) en vez de mirar los 99 números uno a uno.
 */
public class Ej3_7_Multiplos7 {

    public static void main(String[] args) {
        for (int multiplo = 7; multiplo < 100; multiplo = multiplo + 7) {
            System.out.println(multiplo);
        }
    }
}
```

3.8 Media de diez números

Palabras clave: `#media` `#acumulador` `#diez-numeros`

Idea: `for` de 10 sumando; media fuera con `(double) suma / CANTIDAD` para no perder decimales.

Código completo · `Ej3_8_MediaDiez.java`

```
import java.util.Scanner;

/*
 * 3.8 Pedir diez números enteros y mostrar la media.
 *
 * Idea: for de 10 vueltas acumulando en suma. Al dividir pongo 10.0 (decimal)
 * para que la media no pierda los decimales.
 */
public class Ej3_8_MediaDiez {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        final int CANTIDAD = 10;
        int numero, suma = 0;

        for (int i = 1; i <= CANTIDAD; i++) {
            System.out.print("Número " + i + ": ");
            numero = sc.nextInt();
            suma = suma + numero;
        }
```

```
        System.out.println("Media: " + (double) suma / CANTIDAD);
        sc.close();
    }
}
```

3.9 Tabla de multiplicar con rango

Palabras clave: `#tabla-multiplicar` `#validar` `#rango`

Idea: validar con `do-while` (se repite **mientras esté mal**) y mostrar con un `for` del 1 al 10.

Código completo · `Ej3_9_TablaMultiplicar.java`

```java
import java.util.Scanner;

/*
 * 3.9 Pedir un número entre 1 y 10 (asegurando que está en el rango) y mostrar su tabla de multiplicar.
 *
 * Idea: do-while para validar (se repite mientras esté MAL) y luego un for del 1 al 10.
 */
public class Ej3_9_TablaMultiplicar {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numero;

        do {
            System.out.print("Número entre 1 y 10: ");
            numero = sc.nextInt();
        } while (numero < 1 || numero > 10);

        for (int i = 1; i <= 10; i++) {
            System.out.println(numero + " x " + i + " = " + numero * i);
        }

        sc.close();
    }
}
```

3.10 Suma de los 10 primeros impares

Palabras clave: `#impares` `#suma` `#acumulador`

Idea: el impar número i es `2 * i - 1` (1, 3, 5...). Un `for` de 10 vueltas los suma: 100.

Código completo · `Ej3_10_SumaImpares.java`

```java
/*
 * 3.10 Mostrar la suma de los 10 primeros números impares (1 + 3 + 5 + ... + 19).
 *
 * Idea: el impar número i es 2*i - 1 (i=1 -> 1, i=2 -> 3, i=3 -> 5...).
 * Un for de 10 vueltas los va sumando.
 */
public class Ej3_10_SumaImpares {

    public static void main(String[] args) {
        int suma = 0;

        for (int i = 1; i <= 10; i++) {
            suma = suma + (2 * i - 1);
        }
```

```
        System.out.println("La suma de los 10 primeros impares es " + suma);
    }
}
```

3.11 Factorial

Palabras clave: `#factorial` `#producto` `#long`

Idea (producto): empieza en **1** (con 0 todo daría 0) y se multiplica por 2, 3… n. `long` porque con `int` 13! ya no cabe. 0! = 1 sale solo.

Código completo · `Ej3_11_Factorial.java`

```java
import java.util.Scanner;

/*
 * 3.11 Pedir un número y calcular su factorial. 5! = 5 x 4 x 3 x 2 x 1 = 120.
 *
 * Idea (patrón "producto acumulado"): empiezo en 1 (no en 0, que lo anularía todo)
 * y multiplico por 2, 3, ..., n. Uso long porque el factorial crece muy rápido
 * (con int, a partir de 13! ya no cabe).
 */
public class Ej3_11_Factorial {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n;
        long factorial = 1;

        System.out.print("Introduce un número: ");
        n = sc.nextInt();

        if (n < 0) {
            System.out.println("No existe el factorial de un negativo");
        } else {
            for (int i = 2; i <= n; i++) {
                factorial = factorial * i;
            }
            System.out.println(n + "! = " + factorial);
        }

        sc.close();
    }
}
```

3.12 ¿Hay algún suspenso?

Palabras clave: `#bandera` `#notas` `#suspenso`

Idea (bandera): `haySuspenso` empieza en `false` y se pone a `true` en cuanto hay una nota menor que 5. Al final solo se mira la bandera.

Código completo · `Ej3_12_HaySuspenso.java`

```java
import java.util.Scanner;

/*
 * 3.12 Pedir 5 notas y decir al final si hay algún suspenso.
 *
 * Idea (patrón "bandera"): un boolean empieza en false y se pone a true
 * en cuanto aparece una nota menor que 5. Al final solo miro la bandera.
```

```
 */
public class Ej3_12_HaySuspenso {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double nota;
        boolean haySuspenso = false;

        for (int i = 1; i <= 5; i++) {
            System.out.print("Nota " + i + ": ");
            nota = sc.nextDouble();
            if (nota < 5) {
                haySuspenso = true;
            }
        }

        if (haySuspenso) {
            System.out.println("Hay algún suspenso");
        } else {
            System.out.println("No hay suspensos");
        }

        sc.close();
    }
}
```

3.13 Aprobados, condicionados y suspensos

Palabras clave: `#contador` `#notas` `#aprobados`

Idea: tres contadores y un `if (>= 5) / else if (== 4) / else` dentro del `for`.

Código completo · `Ej3_13_AprobadosSuspensos.java`

```
import java.util.Scanner;

/*
 * 3.13 Dadas 6 notas, contar aprobados, condicionados (nota igual a 4) y suspensos.
 *
 * Idea: tres contadores y un if - else if - else dentro del for.
 * El orden importa: primero aprobado (>= 5), luego el caso exacto del 4, y lo demás es
suspenso.
 */
public class Ej3_13_AprobadosSuspensos {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double nota;
        int aprobados = 0, condicionados = 0, suspensos = 0;

        for (int i = 1; i <= 6; i++) {
            System.out.print("Nota " + i + ": ");
            nota = sc.nextDouble();

            if (nota >= 5) {
                aprobados++;
            } else if (nota == 4) {
                condicionados++;
            } else {
                suspensos++;
            }
        }
```

```
        System.out.println("Aprobados: " + aprobados);
        System.out.println("Condicionados: " + condicionados);
        System.out.println("Suspensos: " + suspensos);

        sc.close();
    }
}
```

3.14 Tablas del 1 al 10

Palabras clave: `#tabla-multiplicar` `#for-anidado`

Idea (anidados): el de fuera elige la tabla y el de dentro hace sus 10 líneas: 10 × 10 = 100 líneas.

Código completo · `Ej3_14_TablasDel1Al10.java`

```java
/*
 * 3.14 Mostrar las tablas de multiplicar del 1 al 10.
 *
 * Idea (bucles anidados): el for de fuera elige la tabla y el de dentro hace sus 10 líneas.
 * Por cada vuelta del de fuera, el de dentro se recorre entero.
 */
public class Ej3_14_TablasDel1Al10 {

    public static void main(String[] args) {
        for (int tabla = 1; tabla <= 10; tabla++) {
            System.out.println("Tabla del " + tabla);
            for (int i = 1; i <= 10; i++) {
                System.out.println(tabla + " x " + i + " = " + tabla * i);
            }
            System.out.println();
        }
    }
}
```

3.15 Triángulo de asteriscos

Palabras clave: `#triangulo` `#figura` `#asteriscos`

Idea: el `for` de fuera va **de n bajando a 1**; el de dentro pinta tantos `*` como la fila. `print` dentro y `println()` al acabar la fila. **Para cambiarlo:** al revés (`*`, `**`, `***`) con `for (fila = 1; fila <= n; fila++)`.

Código completo · `Ej3_15_TrianguloAsteriscos.java`

```java
import java.util.Scanner;

/*
 * 3.15 Pedir n y dibujar un triángulo rectángulo de n asteriscos de lado. Para n = 4:
 * ****
 * ***
 * **
 * *
 *
 * Idea: el for de fuera recorre las filas empezando en n y bajando hasta 1;
 * el de dentro pinta tantos asteriscos como indique la fila.
 */
public class Ej3_15_TrianguloAsteriscos {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        int n;

        System.out.print("Lado del triángulo: ");
        n = sc.nextInt();

        for (int fila = n; fila >= 1; fila--) {
            for (int i = 1; i <= fila; i++) {
                System.out.print("*");
            }
            System.out.println();
        }

        sc.close();
    }
}
```

UD3 II. Bucles con cuentas (3.11 a 3.20)

El truco en casi todos: **hacer la cuenta a mano con un ejemplo pequeño** y luego pasar esos pasos al bucle.

3.11 Decimal a binario

Palabras clave: `#binario` `#decimal-a-binario` `#potencias`

Idea: en binario cada posición vale una potencia de 2 (... 8, 4, 2, 1). Se busca la más grande que cabe (doblando desde 1) y se baja: si cabe en lo que queda, escribo 1 y la resto; si no, 0.

Potencia	Queda	¿Cabe?	Escribo
8	13	Sí	1 (quedan 5)
4	5	Sí	1 (queda 1)
2	1	No	0
1	1	Sí	1

13 = **1101**, en el orden correcto (dividiendo entre 2 los restos salen al revés).

Código completo · `Ej3_11_DecimalABinario.java`

```
import java.util.Scanner;

/*
 * 3.11 Convertir un número decimal a binario, sin saber cuántas cifras tiene.
 * Por simplicidad, se muestra un dígito por línea.
 *
 * Idea: el binario se lee de izquierda a derecha con las potencias de 2 (..., 8, 4, 2, 1).
 *   1) Busco la potencia de 2 más grande que cabe en el número (empiezo en 1 y voy
doblando).
 *   2) Voy bajando de potencia en potencia: si la potencia cabe en lo que queda, escribo 1 y
la resto;
 *      si no cabe, escribo 0.
 * Ejemplo 13: potencias 8, 4, 2, 1 -> 8 cabe (1, quedan 5), 4 cabe (1, queda 1), 2 no (0), 1
cabe (1) = 1101
 * (Así salen en orden. Si se divide entre 2 los restos salen al revés.)
 */
public class Ej3_11_DecimalABinario {

    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
        int numero, potencia = 1;

        System.out.print("Número decimal (positivo): ");
        numero = sc.nextInt();

        if (numero < 0) {
            System.out.println("Tiene que ser positivo");
        } else {
            // 1) Potencia de 2 más grande que no se pasa del número
            while (potencia * 2 <= numero) {
                potencia = potencia * 2;
            }

            // 2) Bajo de potencia en potencia escribiendo 1 o 0
            while (potencia >= 1) {
                if (numero >= potencia) {
                    System.out.println(1);
                    numero = numero - potencia;
                } else {
                    System.out.println(0);
                }
                potencia = potencia / 2;
            }
        }

        sc.close();
    }
}
```

3.12 Binario a decimal

Palabras clave: `#binario` `#binario-a-decimal` `#cifras`

Idea: leer el binario como número normal y recorrerlo de derecha a izquierda con `% 10` y `/ 10`. Cada cifra vale 1, 2, 4, 8... (la potencia se dobla). 1101 → 1·1 + 0·2 + 1·4 + 1·8 = 13. Una bandera `valido` detecta cifras mayores que 1.

Código completo · `Ej3_12_BinarioADecimal.java`

```
import java.util.Scanner;

/*
 * 3.12 El usuario introduce un número en binario y el programa lo pasa a decimal.
 *
 * Idea: leo el binario como un entero normal (por ejemplo 1101) y lo recorro de derecha a
izquierda:
 *   - binario % 10 -> la última cifra (0 o 1)
 *   - binario / 10 -> quita la última cifra
 * La cifra de más a la derecha vale 1, la siguiente 2, luego 4, 8... (la potencia se va
doblando).
 * 1101 -> 1*1 + 0*2 + 1*4 + 1*8 = 13
 * Si alguna cifra no es 0 ni 1, el número no es binario.
 */
public class Ej3_12_BinarioADecimal {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int binario, cifra, decimal = 0, potencia = 1;
        boolean valido = true;

        System.out.print("Número binario: ");
        binario = sc.nextInt();
```

```
        while (binario > 0 && valido) {
            cifra = binario % 10;
            if (cifra > 1) {
                valido = false;
            } else {
                decimal = decimal + cifra * potencia;
                potencia = potencia * 2;
                binario = binario / 10;
            }
        }

        if (valido) {
            System.out.println("En decimal: " + decimal);
        } else {
            System.out.println("Eso no es un número binario (solo 0 y 1)");
        }

        sc.close();
    }
}
```

3.13 Incrementar un reloj

Palabras clave: `#hora` `#reloj` `#sumar-segundos`

Idea: pasarlo todo a segundos, sumar y volver a separar:

1. `total = hora * 3600 + minuto * 60 + segundo + incremento;`
2. `total = total % 86400;` (segundos de un día: después de 23:59:59 vuelve a 00:00:00)
3. `hora = total / 3600;` · `minuto = total % 3600 / 60;` · `segundo = total % 60;`

13:59:51 + 10 → 14:00:01.

Código completo · `Ej3_13_IncrementarReloj.java`

```
import java.util.Scanner;

/*
 * 3.13 Pedir hora, minutos y segundos, y cuántos segundos se quiere avanzar.
 * Mostrar la nueva hora. Ejemplo: 13:59:51 + 10 s -> 14:00:01.
 *
 * Idea (la más sencilla y que sirve para cualquier cantidad de segundos):
 *   1) Paso todo a segundos: total = hora*3600 + minuto*60 + segundo + incremento
 *   2) % 86400 (segundos de un día) para que después de las 23:59:59 vuelva a las 00:00:00
 *   3) Lo vuelvo a separar: hora = total / 3600, minuto = (total % 3600) / 60, segundo =
total % 60
 */
public class Ej3_13_IncrementarReloj {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        final int SEGUNDOS_DIA = 24 * 60 * 60; // 86400
        int hora, minuto, segundo, incremento, total;

        System.out.print("Hora: ");
        hora = sc.nextInt();
        System.out.print("Minutos: ");
        minuto = sc.nextInt();
        System.out.print("Segundos: ");
        segundo = sc.nextInt();
        System.out.print("Segundos a incrementar: ");
        incremento = sc.nextInt();
```

```
        total = hora * 3600 + minuto * 60 + segundo + incremento;
        total = total % SEGUNDOS_DIA;

        hora = total / 3600;
        minuto = total % 3600 / 60;
        segundo = total % 60;

        System.out.println("Nueva hora: "
                + (hora < 10 ? "0" : "") + hora + ":"
                + (minuto < 10 ? "0" : "") + minuto + ":"
                + (segundo < 10 ? "0" : "") + segundo);

        sc.close();
    }
}
```

3.14 Contar primos del 1 al n

Palabras clave: `#primos` `#bandera` `#contador`

Idea: primo = nadie entre 2 y él menos 1 lo divide. Bucle de fuera para los números y bucle de dentro buscando divisor; si lo encuentra, `esPrimo = false` y para. La bandera vuelve a `true` al empezar cada número. El ejemplo del enunciado cuenta el 1 como primo (n = 8 → 5), y el programa hace lo mismo; en matemáticas el 1 no es primo.

Código completo · `Ej3_14_ContarPrimos.java`

```java
import java.util.Scanner;

/*
 * 3.14 Pedir n y decir cuántos números primos hay entre 1 y n.
 * Primo = solo se puede dividir entre 1 y entre él mismo.
 * El ejemplo del enunciado (n = 8) cuenta el 1 como primo: 1, 2, 3, 5, 7 -> 5 primos.
 * (En matemáticas el 1 no se considera primo; lo contamos para que coincida con el
enunciado.)
 *
 * Idea: for de 1 a n y, para cada número, otro bucle que busca algún divisor entre 2 y
número-1.
 * Si encuentra uno, no es primo (bandera esPrimo = false) y deja de buscar.
 */
public class Ej3_14_ContarPrimos {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n, primos = 0;
        boolean esPrimo;

        System.out.print("Introduce n: ");
        n = sc.nextInt();

        for (int numero = 1; numero <= n; numero++) {
            esPrimo = true;
            int divisor = 2;
            while (divisor < numero && esPrimo) {
                if (numero % divisor == 0) {
                    esPrimo = false;
                }
                divisor++;
            }

            if (esPrimo) {
```

```
                System.out.println(numero + " -> primo");
                primos++;
            } else {
                System.out.println(numero + " -> no primo");
            }
        }

        System.out.println("Total: " + primos + " números primos");
        sc.close();
    }
}
```

3.15 Triángulo de Pascal

Palabras clave: `#pascal` `#triangulo` `#factorial`

Idea: calcular tres factoriales por número funciona, pero es largo y se desborda. Simplificando la misma fórmula: cada fila empieza en 1 y **cada número sale del anterior**: `valor = valor * (fila - m) / (m + 1)`. Fila 4: 1 → 4 → 6 → 4 → 1. Si piden la fórmula literal, los factoriales se calculan con el bucle del 3.11.

Código completo · `Ej3_15_TrianguloPascal.java`

```java
import java.util.Scanner;

/*
 * 3.15 Dibujar el triángulo de Pascal de n filas. Fórmula del elemento m de la fila n:
 *          E(n, m) = n! / (m! * (n - m)!)
 * Ejemplo (5 filas):
 * 1
 * 1 1
 * 1 2 1
 * 1 3 3 1
 * 1 4 6 4 1
 *
 * Idea: calcular tres factoriales para cada número funciona, pero es lento y con int
 * se desborda enseguida. Simplificando la misma fórmula sale un truco muy fácil:
 * en cada fila el primer número es 1, y cada número se saca del anterior así:
 *          siguiente = anterior * (fila - m) / (m + 1)
 * Fila 4: 1 -> 1*4/1 = 4 -> 4*3/2 = 6 -> 6*2/3 = 4 -> 4*1/4 = 1
 */
public class Ej3_15_TrianguloPascal {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int filas;
        long valor;

        System.out.print("Número de filas: ");
        filas = sc.nextInt();

        for (int fila = 0; fila < filas; fila++) {
            valor = 1; // cada fila empieza en 1
            for (int m = 0; m <= fila; m++) {
                System.out.print(valor + " ");
                valor = valor * (fila - m) / (m + 1);
            }
            System.out.println();
        }

        sc.close();
```

```
    }
}
```

3.16 Triángulo centrado

Palabras clave: `#triangulo` `#centrado` `#figura`

Idea: cada asterisco es `"* "` (ocupa 2 huecos). En la fila f: `n - f` espacios delante y `f` veces `"* "`.

Código completo · `Ej3_16_TrianguloCentrado.java`

```java
import java.util.Scanner;

/*
 * 3.16 Pedir n y dibujar un triángulo de base y altura n. Para n = 4:
 *    *
 *   * *
 *  * * *
 * * * * *
 *
 * Idea: cada asterisco se escribe como "* " (asterisco y espacio), así ocupa 2 huecos.
 * En la fila f (de 1 a n) van f asteriscos y, delante, n - f espacios para centrarlo.
 */
public class Ej3_16_TrianguloCentrado {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n;

        System.out.print("Introduce n: ");
        n = sc.nextInt();

        for (int fila = 1; fila <= n; fila++) {
            for (int i = 1; i <= n - fila; i++) {
                System.out.print(" ");
            }
            for (int i = 1; i <= fila; i++) {
                System.out.print("* ");
            }
            System.out.println();
        }

        sc.close();
    }
}
```

3.17 Máximo común divisor

Palabras clave: `#mcd` `#divisor`

Idea: empezar en el menor (`a < b ? a : b`) y bajar **mientras no divida a los dos** (`a % mcd != 0 || b % mcd != 0`). Como mucho llega a 1. MCD(12, 18) = 6.

Código completo · `Ej3_17_MCD.java`

```java
import java.util.Scanner;

/*
 * 3.17 Máximo común divisor (el número más grande que divide a los dos).
 * Método sencillo: desde el menor de a y b, ir bajando hasta encontrar
 * el primer número que divide a ambos.
```

```
 *
 * Idea: empiezo en el menor y bajo de 1 en 1. El while para en cuanto
 * el número divide a los dos (resto 0 en ambos). Como mucho llega a 1, que divide a todo.
 */
public class Ej3_17_MCD {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a, b, mcd;

        System.out.print("Primer número: ");
        a = sc.nextInt();
        System.out.print("Segundo número: ");
        b = sc.nextInt();

        if (a <= 0 || b <= 0) {
            System.out.println("Los números tienen que ser positivos");
        } else {
            mcd = a < b ? a : b; // el menor de los dos
            while (a % mcd != 0 || b % mcd != 0) {
                mcd--;
            }
            System.out.println("MCD(" + a + ", " + b + ") = " + mcd);
        }

        sc.close();
    }
}
```

3.18 Mínimo común múltiplo

Palabras clave: `#mcm` `#multiplos`

Idea: el mcm es múltiplo del mayor: probar mayor, 2·mayor, 3·mayor… hasta que se pueda dividir entre el menor. MCM(4, 6): 6 no, 12 sí → 12.

Código completo · `Ej3_18_MCM.java`

```
import java.util.Scanner;

/*
 * 3.18 Mínimo común múltiplo (el número más pequeño que es múltiplo de los dos),
 * con un método parecido al 3.17.
 *
 * Idea: el mcm es un múltiplo del mayor. Así que pruebo los múltiplos del mayor
 * (mayor, 2*mayor, 3*mayor...) hasta encontrar el primero que también se pueda dividir
 * entre el menor. Como mucho llega a a*b.
 */
public class Ej3_18_MCM {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a, b, mayor, menor, mcm;

        System.out.print("Primer número: ");
        a = sc.nextInt();
        System.out.print("Segundo número: ");
        b = sc.nextInt();

        if (a <= 0 || b <= 0) {
            System.out.println("Los números tienen que ser positivos");
        } else {
            mayor = a > b ? a : b;
```

```
            menor = a > b ? b : a;

            mcm = mayor;
            while (mcm % menor != 0) {
                mcm = mcm + mayor;
            }
            System.out.println("MCM(" + a + ", " + b + ") = " + mcm);
        }

        sc.close();
    }
}
```

3.19 Raíz cuadrada por aproximación

Palabras clave: `#raiz` `#raiz-cuadrada` `#aproximacion`

Idea: subir la raíz **mientras el cuadrado del siguiente no se pase**: `while ((raiz + 1) * (raiz + 1) <= numero) raiz++;`. Resto = número – raíz². 23 → raíz 4, resto 7.

Código completo · `Ej3_19_RaizAproximada.java`

```
import java.util.Scanner;

/*
 * 3.19 Raíz cuadrada entera de un número natural por aproximaciones, y el resto si no es
exacta.
 * Ejemplo 23: 1²=1, 2²=4, 3²=9, 4²=16, 5²=25 (me paso) -> raíz 4, resto 23 - 16 = 7.
 *
 * Idea: voy subiendo raiz mientras el cuadrado del SIGUIENTE no se pase del número.
 * Al salir, raiz es el último que cabía y resto = numero - raiz*raiz.
 */
public class Ej3_19_RaizAproximada {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numero, raiz = 0, resto;

        System.out.print("Número natural: ");
        numero = sc.nextInt();

        if (numero < 0) {
            System.out.println("Tiene que ser un número natural (0 o positivo)");
        } else {
            while ((raiz + 1) * (raiz + 1) <= numero) {
                raiz++;
            }
            resto = numero - raiz * raiz;

            if (resto == 0) {
                System.out.println("La raíz de " + numero + " es exacta: " + raiz);
            } else {
                System.out.println("La raíz de " + numero + " es " + raiz + " con resto " +
resto);
            }
        }

        sc.close();
    }
}
```

3.20 Suma de dinero

Palabras clave: `#centinela` `#acumulador` `#dinero`

Idea: centinela (el 0) + acumulador. `double` porque el dinero lleva céntimos.

Código completo · `Ej3_20_SumaDinero.java`

```java
import java.util.Scanner;

/*
 * 3.20 Pedir las cantidades de dinero que tiene el usuario (banco, hucha, cajón,
bolsillos...)
 * hasta que introduzca 0, y mostrar la suma total.
 *
 * Idea: centinela (el 0) + acumulador. Uso double porque el dinero lleva céntimos.
 */
public class Ej3_20_SumaDinero {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double cantidad, total = 0;

        System.out.print("Cantidad (0 para terminar): ");
        cantidad = sc.nextDouble();

        while (cantidad != 0) {
            total = total + cantidad;
            System.out.print("Cantidad (0 para terminar): ");
            cantidad = sc.nextDouble();
        }

        System.out.println("Dinero total: " + total + " euros");
        sc.close();
    }
}
```

Tema 3. Ejercicios adicionales (26 a 33)

Las figuras se resuelven igual: **`for` de filas y, dentro, el que decide qué va en cada hueco**. Dibuja la figura en papel cuadriculado, numera filas y columnas y busca la regla.

26. Primos hasta N

Palabras clave: `#primos` `#validar`

Idea: como el 3.14, empezando en 2. Mejoras: validar N > 1 con `do-while` y buscar divisores **solo hasta la raíz** (`divisor * divisor <= numero`): si 36 tuviera un divisor mayor que 6, tendría otro menor que ya habríamos encontrado. N = 30 → 2 3 5 7 11 13 17 19 23 29.

Código completo · `Ej26_PrimosHastaN.java`

```java
import java.util.Scanner;

/*
 * Ejercicio 26. Pedir un número N mayor que 1 y mostrar los números primos entre 1 y N.
 *
 * Idea: igual que el 3.14 pero mostrando solo los primos (aquí el 1 no cuenta, como en
matemáticas).
 * Truco para ir más rápido: basta con buscar divisores hasta la raíz del número
 * (divisor * divisor <= numero). Si 36 tuviera un divisor mayor que 6, tendría otro menor
```

```
que 6.
 */
public class Ej26_PrimosHastaN {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n;
        boolean esPrimo;

        do {
            System.out.print("Introduce N (mayor que 1): ");
            n = sc.nextInt();
        } while (n <= 1);

        System.out.print("Primos entre 1 y " + n + ": ");
        for (int numero = 2; numero <= n; numero++) {
            esPrimo = true;
            for (int divisor = 2; divisor * divisor <= numero && esPrimo; divisor++) {
                if (numero % divisor == 0) {
                    esPrimo = false;
                }
            }
            if (esPrimo) {
                System.out.print(numero + " ");
            }
        }
        System.out.println();

        sc.close();
    }
}
```

27. Cuadrado de asteriscos

Palabras clave: `#cuadrado` `#figura`

Idea: dos `for` anidados del mismo tamaño; cada asterisco `"* "`. Un `do-while` asegura el lado entre 1 y 10.

Código completo · `Ej27_Cuadrado.java`

```
import java.util.Scanner;

/*
 * Ejercicio 27. Pedir iNumLados entre 1 y 10 y pintar un cuadrado de * de ese lado.
 * Para 4:
 * * * * *
 * * * * *
 * * * * *
 * * * * *
 *
 * Idea: dos for anidados del mismo tamaño: filas y columnas. Cada * se escribe "* ".
 */
public class Ej27_Cuadrado {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int iNumLados;

        do {
            System.out.print("Introduzca longitud del lado (1-10): ");
            iNumLados = sc.nextInt();
        } while (iNumLados < 1 || iNumLados > 10);
```

```
        for (int fila = 1; fila <= iNumLados; fila++) {
            for (int col = 1; col <= iNumLados; col++) {
                System.out.print("* ");
            }
            System.out.println();
        }

        sc.close();
    }
}
```

28. Cuadrado hueco

Palabras clave: `#cuadrado` `#cuadrado-hueco` `#borde` `#figura`

Idea: asterisco solo en el **borde** (`fila == 1 || fila == lado || col == 1 || col == lado`); en el resto dos espacios. **Para cambiarlo:** una X dentro con `|| fila == col || fila + col == lado + 1`.

Código completo · `Ej28_CuadradoHueco.java`

```
import java.util.Scanner;

/*
 * Ejercicio 28. Como el 27, pero el cuadrado tiene que quedar hueco. Para 6:
 * * * * * * *
 * *         *
 * *         *
 * *         *
 * *         *
 * * * * * * *
 *
 * Idea: se pinta * solo en el BORDE: primera o última fila, o primera o última columna.
 * En el resto se pintan espacios ("  ", dos para que ocupe lo mismo que "* ").
 */
public class Ej28_CuadradoHueco {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int lado;

        do {
            System.out.print("Introduzca longitud del lado (1-10): ");
            lado = sc.nextInt();
        } while (lado < 1 || lado > 10);

        for (int fila = 1; fila <= lado; fila++) {
            for (int col = 1; col <= lado; col++) {
                if (fila == 1 || fila == lado || col == 1 || col == lado) {
                    System.out.print("* ");
                } else {
                    System.out.print("  ");
                }
            }
            System.out.println();
        }

        sc.close();
    }
}
```

29. O digital

Palabras clave: `#cuadrado` `#o-digital` `#figura`

Idea: tres partes por separado: arriba (un espacio y `lado` veces `"* "`), los lados (`lado` filas con `*`, espacios, `*`) y abajo (igual que arriba). Partirlo en trozos suele ser más fácil que una fórmula para todo.

Código completo · `Ej29_ODigital.java`

```java
import java.util.Scanner;

/*
 * Ejercicio 29. Variación del 28: convertir el cuadrado hueco en una "O digital"
 * (las esquinas no se pintan, así parece un número 0 de un reloj digital). Para 6:
 *  * * * * * *
 * *           *
 * *           *
 * *           *
 * *           *
 * *           *
 * *           *
 *  * * * * * *
 *
 * Idea: tres partes.
 *   - Arriba: un espacio y luego "lado" veces "* ".
 *   - Lados: "lado" filas con un * a la izquierda, huecos y un * a la derecha.
 *   - Abajo: igual que arriba.
 */
public class Ej29_ODigital {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int lado;

        System.out.print("Introduzca tamaño del lado: ");
        lado = sc.nextInt();

        // Línea de arriba
        System.out.print(" ");
        for (int i = 1; i <= lado; i++) {
            System.out.print("* ");
        }
        System.out.println();

        // Lados
        for (int fila = 1; fila <= lado; fila++) {
            System.out.print("*");
            for (int i = 1; i <= lado * 2 - 1; i++) {
                System.out.print(" ");
            }
            System.out.println("*");
        }

        // Línea de abajo
        System.out.print(" ");
        for (int i = 1; i <= lado; i++) {
            System.out.print("* ");
        }
        System.out.println();

        sc.close();
```

```
    }
}
```

30. Rombo

Palabras clave: `#rombo` `#figura`

Idea: `2·lado - 1` filas. Cada fila: espacios por fuera + `*` + espacios por dentro + `*`. Con la **distancia a la fila central** `d` (0 en el centro, lado-1 en las puntas) no hace falta repetir código: fuera = `d`, dentro = `2·(lado - 1 - d) - 1`. Si dentro sale 0 o menos, es la punta y lleva un solo `*`.

Fila (lado 3)	d	Fuera	Dentro
0	2	2	— (punta)
1	1	1	1
2	0	0	3
3	1	1	1
4	2	2	— (punta)

Código completo · `Ej30_Rombo.java`

```java
import java.util.Scanner;

/*
 * Ejercicio 30. Dibujar un rombo hueco con * de lado n. Para 3:
 *   *
 *  * *
 * *   *
 *  * *
 *   *
 *
 * Idea: el rombo tiene 2*lado - 1 filas. En cada fila pinto:
 *   espacios por fuera + "*" + espacios por dentro + "*" (la fila de la punta solo lleva un
*).
 * Para no repetir el código de la mitad de arriba y la de abajo, uso la "distancia a la
punta":
 *   d = cuánto se aleja la fila de la fila central (0 en el centro, lado-1 en las puntas).
 *   - espacios por fuera = d
 *   - espacios por dentro = 2 * (lado - 1 - d) - 1
 */
public class Ej30_Rombo {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int lado, d, interior;

        System.out.print("Introduzca tamaño del lado: ");
        lado = sc.nextInt();

        for (int fila = 0; fila < 2 * lado - 1; fila++) {
            // distancia de esta fila a la fila central (lado - 1)
            d = fila - (lado - 1);
            if (d < 0) {
                d = -d;
            }
            interior = 2 * (lado - 1 - d) - 1;

            for (int i = 0; i < d; i++) {
```

```
                System.out.print(" ");
            }
            System.out.print("*");
            if (interior > 0) {
                for (int i = 0; i < interior; i++) {
                    System.out.print(" ");
                }
                System.out.print("*");
            }
            System.out.println();
        }

        sc.close();
    }
}
```

31. Tabla de rombos

Palabras clave: `#rombo` `#tabla-rombos` `#figura`

La clave: la consola escribe **línea a línea**, así que se pinta la línea 1 de todos los rombos de la fila, luego la línea 2… Cada trozo se rellena con espacios hasta el ancho del rombo (`2·lado - 1`) para que el siguiente empiece siempre en la misma columna.

Código completo · `Ej31_TablaRombos.java`

```
import java.util.Scanner;

/*
 * Ejercicio 31. Mejora del 30: pedir lado n, filas y columnas, y dibujar una tabla
 * de filas x columnas rombos de lado n.
 *
 * Idea: en la consola solo se escribe línea a línea, así que NO se puede pintar un rombo
entero
 * y luego el de al lado. Hay que pintar la línea 1 de todos los rombos de la fila, luego la
línea 2...
 *   - for de filas de rombos
 *       - for de líneas del rombo (2*lado - 1)
 *           - for de columnas: pinto ESA línea del rombo, rellenando con espacios hasta su
ancho
 *             (2*lado - 1) para que el siguiente rombo empiece siempre en el mismo sitio
 *       - línea en blanco entre filas de rombos
 */
public class Ej31_TablaRombos {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int lado, filas, columnas, d, interior, ancho, escritos;

        System.out.print("Introduzca tamaño del lado: ");
        lado = sc.nextInt();
        System.out.print("Introduzca filas: ");
        filas = sc.nextInt();
        System.out.print("Introduzca columnas: ");
        columnas = sc.nextInt();

        ancho = 2 * lado - 1;

        for (int filaRombos = 1; filaRombos <= filas; filaRombos++) {
            for (int linea = 0; linea < ancho; linea++) {
                d = linea - (lado - 1);
                if (d < 0) {
                    d = -d;
```

```
                }
                interior = 2 * (lado - 1 - d) - 1;

                for (int col = 1; col <= columnas; col++) {
                    escritos = 0; // caracteres escritos de este rombo en esta línea
                    for (int i = 0; i < d; i++) {
                        System.out.print(" ");
                        escritos++;
                    }
                    System.out.print("*");
                    escritos++;
                    if (interior > 0) {
                        for (int i = 0; i < interior; i++) {
                            System.out.print(" ");
                            escritos++;
                        }
                        System.out.print("*");
                        escritos++;
                    }
                    // relleno hasta el ancho del rombo + 1 espacio de separación
                    while (escritos <= ancho) {
                        System.out.print(" ");
                        escritos++;
                    }
                }
                System.out.println();
            }
            System.out.println();
        }

        sc.close();
    }
}
```

32. Decimal a hexadecimal

Palabras clave: `#hexadecimal` `#decimal-a-hex` `#charAt`

Idea: como a binario pero entre 16. `numero % 16` da la cifra (0-15 → "0123456789ABCDEF") y `numero / 16` pasa a la siguiente. Salen de derecha a izquierda, así que cada cifra nueva va **delante**: `hex = cifra + hex`. 255 → FF. El 0 se trata aparte.

Código completo · `Ej32_DecimalAHex.java`

```
import java.util.Scanner;

/*
 * Ejercicio 32. Pasar un número decimal a hexadecimal sin usar funciones de librería.
 * Hexadecimal = base 16. Cifras: 0-9 y luego A=10, B=11, C=12, D=13, E=14, F=15.
 *
 * Idea (igual que a binario pero dividiendo entre 16):
 *   - resto = numero % 16 -> la última cifra hexadecimal
 *   - numero = numero / 16 -> pasa a la siguiente
 * Los restos salen de derecha a izquierda, así que cada cifra nueva se pone DELANTE del
texto:
 *   hex = cifra + hex
 * Ejemplo 255: 255 % 16 = 15 (F), 255 / 16 = 15 -> 15 % 16 = 15 (F) -> "FF"
 */
public class Ej32_DecimalAHex {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        final String CIFRAS_HEX = "0123456789ABCDEF";
```

```
        int numero, resto;
        String hex = "";

        System.out.print("Número decimal (positivo): ");
        numero = sc.nextInt();

        if (numero < 0) {
            System.out.println("Tiene que ser positivo");
        } else if (numero == 0) {
            System.out.println("En hexadecimal: 0");
        } else {
            while (numero > 0) {
                resto = numero % 16;
                hex = CIFRAS_HEX.charAt(resto) + hex; // la cifra va delante
                numero = numero / 16;
            }
            System.out.println("En hexadecimal: " + hex);
        }

        sc.close();
    }
}
```

33. Hexadecimal a decimal

Palabras clave: `#hexadecimal` `#hex-a-decimal` `#charAt` `#texto`

Idea: leer como texto y recorrer con `charAt`: `decimal = decimal * 16 + valor`. Valor del carácter: `'0'`–`'9'` → `c - '0'`; `'A'`–`'F'` → `c - 'A' + 10`; otra cosa → no válido. `.toUpperCase()` acepta `ff` igual que `FF`. 1A → 26.

Código completo · `Ej33_HexADecimal.java`

```
import java.util.Scanner;

/*
 * Ejercicio 33. Lo contrario del 32: pedir un número hexadecimal y mostrarlo en decimal.
 *
 * Idea: lo leo como texto (String) y lo recorro de izquierda a derecha con charAt.
 * Por cada cifra: decimal = decimal * 16 + valorDeLaCifra
 * (es como leer "345" en decimal: ((3*10)+4)*10+5, pero multiplicando por 16).
 * Valor de la cifra: si es '0'-'9' -> c - '0'; si es 'A'-'F' -> c - 'A' + 10.
 * Las letras se pasan antes a mayúsculas para aceptar "ff" igual que "FF".
 */
public class Ej33_HexADecimal {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String hex;
        char c;
        int valor, decimal = 0;
        boolean valido = true;

        System.out.print("Número hexadecimal: ");
        hex = sc.next().toUpperCase();

        for (int i = 0; i < hex.length() && valido; i++) {
            c = hex.charAt(i);
            if (c >= '0' && c <= '9') {
                valor = c - '0';
            } else if (c >= 'A' && c <= 'F') {
                valor = c - 'A' + 10;
            } else {
```

```
                valor = 0;
                valido = false;
            }
            decimal = decimal * 16 + valor;
        }

        if (valido) {
            System.out.println("En decimal: " + decimal);
        } else {
            System.out.println("No es un número hexadecimal válido");
        }

        sc.close();
    }
}
```

Parte 3. Exámenes de otros años resueltos

Son los exámenes reales de cursos anteriores. Lo que cae suele parecerse mucho, así que aquí va cada ejercicio con el enunciado resumido, la idea y el código.

Examen práctico Temas 1, 2 y 3 (08-10-2025)

Ejercicio 1. Conversión de temperaturas

Palabras clave: `#temperaturas` `#conversion` `#celsius` `#rango`

Enunciado: pedir grados Celsius y mostrar Fahrenheit y Kelvin (variables `double`). Solo vale entre -100 y 3000 (incluidos); si no, termina. F = C · 9/5 + 32; K = C + 273.15.

Idea: un `if` para el rango y, si está bien, aplicar las dos fórmulas. Cuidado: `9 / 5` entre enteros da 1; por eso se escribe `celsius * 9 / 5` (el `double` va primero y no se pierden decimales).

Código completo · `Ejercicio1Temperaturas.java`

```java
import java.util.Scanner;

public class Ejercicio1Temperaturas {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double celsius, fahrenheit, kelvin;

        System.out.print("Introduce la temperatura en grados Celsius: ");
        celsius = sc.nextDouble();

        // Solo se aceptan temperaturas entre -100 y 3000 (incluidos)
        if (celsius < -100 || celsius > 3000) {
            System.out.println("Temperatura fuera de rango. Fin del programa");
        } else {
            fahrenheit = (celsius * 9 / 5) + 32;
            kelvin = celsius + 273.15;

            System.out.println(celsius + " ºC son " + fahrenheit + " ºF");
            System.out.println(celsius + " ºC son " + kelvin + " K");
        }
        sc.close();
    }
}
```

Ejercicio 2. Suma de múltiplos de n en un rango

Palabras clave: `#multiplos` `#rango` `#acumulador`

Enunciado: pedir n, min y max y sumar los múltiplos de n entre min y max (incluidos). n=3, 12 a 22 → 12+15+18+21 = 66.

Idea: `for` de min a max y, si `numero % n == 0`, se suma.

Código completo · `Ejercicio2Multiplos.java`

```java
import java.util.Scanner;

public class Ejercicio2Multiplos {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n, min, max, suma = 0;

        System.out.print("Introduce n: ");
        n = sc.nextInt();

        System.out.print("Introduce el mínimo: ");
        min = sc.nextInt();

        System.out.print("Introduce el máximo: ");
        max = sc.nextInt();

        if (n <= 0) {
            System.out.println("n tiene que ser mayor que 0");
        } else {
            // Recorro del mínimo al máximo (incluidos) y sumo los múltiplos de n
            for (int numero = min; numero <= max; numero++) {
                if (numero % n == 0) {
                    suma = suma + numero;
                }
            }
            System.out.println("La suma de los múltiplos de " + n + " entre " + min + " y " +
max + " es " + suma);
        }
        sc.close();
    }
}
```

Ejercicio 3. Matriz formateada n × n

Palabras clave: `#matriz` `#ceros-delante` `#formato` `#cifras`

Enunciado: matriz de n × n con los números de 1 a n², con ceros delante para que todos tengan las cifras de n² (n=4 → 01 02 ... 16; n=10 → 001 ... 100).

Idea: contar las cifras de n² (dividiendo entre 10). Para cada número, contar sus cifras y escribir tantos `0` como le falten.

Código completo · `Ejercicio3Matriz.java`

```java
import java.util.Scanner;

public class Ejercicio3Matriz {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n, total, digitosMax, numero, digitos, aux;

        System.out.print("Introduce n: ");
        n = sc.nextInt();

        total = n * n;

        // Cuento cuántas cifras tiene n*n, que es el número más grande
        digitosMax = 0;
        aux = total;
        while (aux > 0) {
```

```
            aux = aux / 10;
            digitosMax++;
        }

        numero = 1;
        for (int fila = 1; fila <= n; fila++) {
            for (int columna = 1; columna <= n; columna++) {

                // Cuento las cifras del número que voy a escribir
                digitos = 0;
                aux = numero;
                while (aux > 0) {
                    aux = aux / 10;
                    digitos++;
                }

                // Pongo los 0 que le faltan delante
                for (int i = digitos; i < digitosMax; i++) {
                    System.out.print("0");
                }

                System.out.print(numero + " ");
                numero++;
            }
            System.out.println();
        }
        sc.close();
    }
}
```

Ejercicio 4. Encriptación base 2 (menú)

Palabras clave: `#menu` `#encriptar` `#texto` `#charAt` `#intercambiar-letras`

Enunciado: menú con 1 introducir texto, 2 encriptar, 3 desencriptar, 4 mostrar; cualquier otra cosa sale. Encriptar = cambiar las letras de dos en dos ("hola" → "ohal"); si sobra una, se queda igual.

Idea: menú con `while (!salir)` + `switch`. Encriptar y desencriptar son la misma operación (volver a cambiar las letras). Se recorre el texto de 2 en 2 con `charAt` y se pegan al revés. La opción se lee con `nextLine()` para que cualquier cosa salga sin error.

Código completo · `Ejercicio4Encriptacion.java`

```
import java.util.Scanner;

public class Ejercicio4Encriptacion {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String texto = "", resultado, opcion;
        boolean salir = false;

        while (!salir) {
            System.out.println();
            System.out.println("Programa de encriptado base 2.");
            System.out.println("1. Introducir texto");
            System.out.println("2. Encriptar texto");
            System.out.println("3. Desencriptar texto");
            System.out.println("4. Mostrar texto");
            System.out.println("(Cualquier otra cosa, salir del programa)");
            System.out.print("Opción: ");
```

```
            // Leo la opción como texto para que si meten cualquier cosa no pete
            opcion = sc.nextLine();

            switch (opcion) {
                case "1":
                    System.out.print("Escribe el texto: ");
                    texto = sc.nextLine();
                    break;

                case "2":
                case "3":
                    // Encriptar y desencriptar es lo mismo: volver a cambiar las letras de
sitio
                    if (texto.length() > 1) {
                        resultado = "";
                        int i = 0;
                        // Cojo las letras de dos en dos y las cambio de orden
                        while (i < texto.length() - 1) {
                            resultado = resultado + texto.charAt(i + 1) + texto.charAt(i);
                            i = i + 2;
                        }
                        // Si el texto es impar, la última letra se queda igual
                        if (texto.length() % 2 != 0) {
                            resultado = resultado + texto.charAt(texto.length() - 1);
                        }
                        texto = resultado;

                        if (opcion.equals("2")) {
                            System.out.println("texto encriptado");
                        } else {
                            System.out.println("texto desencriptado");
                        }
                    } else {
                        System.out.println("El texto tiene que tener más de 1 carácter");
                    }
                    break;

                case "4":
                    System.out.println(texto);
                    break;

                default:
                    System.out.println("Fin del programa");
                    salir = true;
            }
        }
        sc.close();
    }
}
```

Ejercicio 5. Reloj de arena

Palabras clave: `#reloj-arena` `#figura` `#puntos`

Enunciado: pedir radio y cantidad y dibujar "relojes de arena" con `.` y `*`. La primera línea tiene radio·2 asteriscos, se pierden 2 por línea y luego se vuelve a abrir. Línea en blanco entre relojes.

Idea: mitad de arriba: en la fila f, `f` puntos + `(radio - f) · 2` asteriscos + `f` puntos. La mitad de abajo es el mismo código con el `for` al revés. Todo dentro de un `for` de relojes.

Código completo · `Ejercicio5RelojArena.java`

```java
import java.util.Scanner;

public class Ejercicio5RelojArena {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int radio, cantidad;

        System.out.print("Introduce el radio: ");
        radio = sc.nextInt();

        System.out.print("Introduce la cantidad: ");
        cantidad = sc.nextInt();

        for (int reloj = 1; reloj <= cantidad; reloj++) {

            // Parte de arriba: en cada línea un punto más a cada lado y 2 asteriscos menos
            for (int fila = 0; fila < radio; fila++) {
                for (int i = 0; i < fila; i++) {
                    System.out.print(".");
                }
                for (int i = 0; i < (radio - fila) * 2; i++) {
                    System.out.print("*");
                }
                for (int i = 0; i < fila; i++) {
                    System.out.print(".");
                }
                System.out.println();
            }

            // Parte de abajo: lo mismo pero al revés (triángulo hacia arriba)
            for (int fila = radio - 1; fila >= 0; fila--) {
                for (int i = 0; i < fila; i++) {
                    System.out.print(".");
                }
                for (int i = 0; i < (radio - fila) * 2; i++) {
                    System.out.print("*");
                }
                for (int i = 0; i < fila; i++) {
                    System.out.print(".");
                }
                System.out.println();
            }

            // Línea en blanco entre relojes (no después del último)
            if (reloj < cantidad) {
                System.out.println();
            }
        }
        sc.close();
    }
}
```

Examen Tema 3 (22-11-2024)

Ejercicio 1. Positivo, negativo, par, hasta repetir

Palabras clave: `#repetir` `#anterior` `#par` `#positivo`

Enunciado (papel): pedir enteros y decir si son positivos, cero o negativos y pares o impares (0 es par). Termina si se mete dos veces seguidas el mismo número.

Idea: guardar el número anterior antes de leer el nuevo y comparar. Una bandera `repetido` controla el bucle.

Código completo · `ej_ex1_1_Numeros.java`

```java
import java.util.Scanner;

/*
 * Ejercicio 1. Realiza en papel, mediante Java, un programa que solicite números enteros
 * consecutivamente y que muestre en pantalla si son positivos, cero o negativos, también
pares o
 * impares (consideramos 0 como par) y que termine si se introduce dos veces seguidas el
mismo
 * número.
 */
public class ej_ex1_1_Numeros {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numero, anterior;
        boolean repetido = false;

        System.out.print("Introduce un número: ");
        numero = sc.nextInt();

        while (!repetido) {
            // Positivo, cero o negativo
            if (numero > 0) {
                System.out.println(numero + " es positivo");
            } else if (numero == 0) {
                System.out.println(numero + " es cero");
            } else {
                System.out.println(numero + " es negativo");
            }

            // Par o impar (el 0 cuenta como par)
            if (numero % 2 == 0) {
                System.out.println(numero + " es par");
            } else {
                System.out.println(numero + " es impar");
            }

            // Guardo el número para compararlo con el siguiente
            anterior = numero;
            System.out.print("Introduce un número: ");
            numero = sc.nextInt();

            if (numero == anterior) {
                repetido = true;
            }
        }

        System.out.println("Has repetido el mismo número dos veces seguidas. Fin del
programa");
        sc.close();
    }
}
```

Ejercicio 2. Movimiento (posición, velocidad, aceleración)

Palabras clave: `#validar` `#movimiento` `#fisica` `#varios-errores`

Enunciado: pedir pos (≥ 0), vel (0-100), acel (5-15) y tiempo (10-100); no seguir hasta que estén bien, diciendo cuáles fallan. Repetir `tiempo` veces: pos = pos + vel; vel = vel + acel. Mostrar los cuatro valores.

Idea: `do-while` con una bandera `correcto` y un `if` por dato, cada uno con su mensaje. Luego un `for` de `tiempo` vueltas.

Código completo · `ej_ex1_2_Movimiento.java`

```java
import java.util.Scanner;

/*
 * Ejercicio 2. Realiza un programa en Java que realice la siguiente función:
 *  - Pedir cuatro números por teclado. Serán posición inicial (pos), velocidad inicial
(vel),
 *    aceleración inicial (acel) y tiempo (tiempo). La posición inicial debe ser un número
positivo
 *    mayor o igual a cero. La velocidad inicial puede ser un número entre 0 y 100, la
aceleración
 *    inicial será un número entre 5 y 15 y el tiempo un número entre 10 y 100 (serán
segundos,
 *    realmente, pero enteros). No se debe de continuar hasta que la entrada de estos cuatro
 *    números cumpla con estos requisitos, informando al usuario si se meten valores erróneos
y
 *    cuáles son.
 *  - Posteriormente se calcularán estas fórmulas tantas veces como nos indique la variable
tiempo:
 *      - pos = pos + vel;
 *      - vel = vel + acel;
 *  - Muestra por pantalla al finalizar el valor final obtenido para las cuatro variables en
el formato:
 *      - Velocidad: … m/s
 *      - Aceleración: … m/s2
 *      - Posición: … m
 *      - Tiempo usado: … segundos
 */
public class ej_ex1_2_Movimiento {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        double pos, vel, acel;
        int tiempo;
        boolean correcto;

        // Pido los cuatro números hasta que estén todos bien
        do {
            System.out.print("Posición inicial: ");
            pos = sc.nextDouble();
            System.out.print("Velocidad inicial: ");
            vel = sc.nextDouble();
            System.out.print("Aceleración inicial: ");
            acel = sc.nextDouble();
            System.out.print("Tiempo (segundos): ");
            tiempo = sc.nextInt();

            correcto = true;

            if (pos < 0) {
                System.out.println("Error: la posición tiene que ser mayor o igual a 0");
```

```
                correcto = false;
            }
            if (vel < 0 || vel > 100) {
                System.out.println("Error: la velocidad tiene que estar entre 0 y 100");
                correcto = false;
            }
            if (acel < 5 || acel > 15) {
                System.out.println("Error: la aceleración tiene que estar entre 5 y 15");
                correcto = false;
            }
            if (tiempo < 10 || tiempo > 100) {
                System.out.println("Error: el tiempo tiene que estar entre 10 y 100");
                correcto = false;
            }

            if (!correcto) {
                System.out.println("Vuelve a introducir los datos");
            }
        } while (!correcto);

        // Repito las fórmulas tantas veces como segundos
        for (int segundo = 1; segundo <= tiempo; segundo++) {
            pos = pos + vel;
            vel = vel + acel;
        }

        System.out.println("Velocidad: " + vel + " m/s");
        System.out.println("Aceleración: " + acel + " m/s2");
        System.out.println("Posición: " + pos + " m");
        System.out.println("Tiempo usado: " + tiempo + " segundos");
        sc.close();
    }
}
```

Ejercicio 3. Matriz con filas negativas

Palabras clave: `#matriz` `#filas-pares` `#negativo`

Enunciado: ancho A y alto B (2 a 10). Cada valor = fila · columna, en negativo si la fila es par.

Idea: validar los dos con `do-while`; dos `for` anidados; `if (fila % 2 == 0) valor = -valor;`; separar con `"\t"`.

Código completo · `ej_ex1_3_Matriz.java`

```
import java.util.Scanner;

/*
 * Ejercicio 3. Realiza una matriz de números de ancho A y altura B que son solicitados por
teclado al
 * usuario, donde A y B deben ser números entre 2 y 10 (obligatoriamente). Cada posición de
la matriz
 * será un número entero calculado mediante la siguiente fórmula:
 *  - Si la fila es par, el número resultado de multiplicar la fila por la columna en
negativo.
 *  - Si la fila es impar, el número resultado de multiplicar la fila por la columna en
positivo.
 *  - Por ejemplo, para A=3 y B=4 la matriz sería:
 *       1  2  3
 *      -2 -4 -6
 *       3  6  9
 *      -4 -8 -12
 */
```

```java
public class ej_ex1_3_Matriz {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int ancho, alto, valor;

        do {
            System.out.print("Ancho A (entre 2 y 10): ");
            ancho = sc.nextInt();
            if (ancho < 2 || ancho > 10) {
                System.out.println("Error: A tiene que estar entre 2 y 10");
            }
        } while (ancho < 2 || ancho > 10);

        do {
            System.out.print("Altura B (entre 2 y 10): ");
            alto = sc.nextInt();
            if (alto < 2 || alto > 10) {
                System.out.println("Error: B tiene que estar entre 2 y 10");
            }
        } while (alto < 2 || alto > 10);

        // Las filas van de 1 a B y las columnas de 1 a A
        for (int fila = 1; fila <= alto; fila++) {
            for (int columna = 1; columna <= ancho; columna++) {
                valor = fila * columna;
                // Si la fila es par, el número va en negativo
                if (fila % 2 == 0) {
                    valor = -valor;
                }
                System.out.print(valor + "\t");
            }
            System.out.println();
        }
        sc.close();
    }
}
```

Ejercicio 4. Árbol de Navidad con + o = U

Palabras clave: `#arbol` `#mitadAncho` `#alternar`

Enunciado: dado mitadAncho, un árbol de ancho mitadAncho·2: primera línea todo `+`, luego líneas alternando `o` y `=` (2, 4, 6… en el centro), y la última todo `+` con `UU` en el centro.

Idea: en la línea k (de 1 a mitadAncho) van `mitadAncho - k` signos `+` a cada lado y `2·k` en el centro. Si k es impar, `o`; si es par, `=`.

Código completo · `ej_ex1_4_Arbol.java`

```java
import java.util.Scanner;

/*
 * Ejercicio 4. Realiza un programa que imprima en pantalla una figura como la siguiente dado un
 * número por teclado que llamaremos mitadAncho. La figura se llama "Árbol de Navidad".
 *
 * Supongamos mitadAncho = 4:
 *   ++++++++
 *   +++oo+++
 *   ++====++
 *   +oooooo+
 *   ========
```

```
 *   +++UU+++
 *
 * Todas las líneas tienen 4*2 -> 8 caracteres. Hay 4 + 2 líneas de "altura". La primera
línea es de
 * símbolos '+'. Después se van alternando líneas con 'o' con '=', terminando en una línea de
todo '+'
 * excepto los dos símbolos centrales que son el tronco formado por 'U'.
 *
 * Realiza el programa que solicite el número mitadAncho (determina cuanto de ancho tiene la
base del
 * árbol) y dibuje el árbol de navidad correspondiente.
 */
public class ej_ex1_4_Arbol {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int mitadAncho;

        do {
            System.out.print("Introduce mitadAncho: ");
            mitadAncho = sc.nextInt();
            if (mitadAncho < 1) {
                System.out.println("Error: tiene que ser mayor que 0");
            }
        } while (mitadAncho < 1);

        // Primera línea: todo '+'
        for (int i = 0; i < mitadAncho * 2; i++) {
            System.out.print("+");
        }
        System.out.println();

        // Copa: en cada línea hay 2 símbolos más en el centro
        for (int linea = 1; linea <= mitadAncho; linea++) {
            // '+' a la izquierda
            for (int i = 0; i < mitadAncho - linea; i++) {
                System.out.print("+");
            }
            // Centro: las líneas impares con 'o' y las pares con '='
            for (int i = 0; i < linea * 2; i++) {
                if (linea % 2 != 0) {
                    System.out.print("o");
                } else {
                    System.out.print("=");
                }
            }
            // '+' a la derecha
            for (int i = 0; i < mitadAncho - linea; i++) {
                System.out.print("+");
            }
            System.out.println();
        }

        // Tronco: todo '+' menos las dos 'U' del centro
        for (int i = 0; i < mitadAncho - 1; i++) {
            System.out.print("+");
        }
        System.out.print("UU");
        for (int i = 0; i < mitadAncho - 1; i++) {
            System.out.print("+");
        }
        System.out.println();
```

```
        sc.close();
    }
}
```

Examen Temas 2 y 3 (08-11-2024)

Ejercicio 1. Dígitos en texto

Palabras clave: `#cifras` `#texto` `#switch` `#digitos`

Enunciado (papel): número de 4 cifras (1000-9999); escribir cada cifra con letras (3428 → "tres cuatro dos ocho"). Si no tiene 4 cifras: "Número no válido."

Idea: sacar las cifras de izquierda a derecha con un divisor que baja 1000 → 100 → 10 → 1 (`numero / divisor % 10`) y un `switch` para el texto.

Código completo · `ej_ex2_1_Digitos.java`

```java
import java.util.Scanner;

/*
 * Ejercicio 1. Realiza en papel, mediante Java, un prototipo de programa que recoja un número de 4
 * cifras entre el 1000 y el 9999, y separe este número en sus cuatro dígitos, para posteriormente
 * escribir en la pantalla cada dígito de forma textual.
 *   - Ejemplo (caso 1):  Introduzca el número: 234   ->  Número no válido.
 *   - Ejemplo (caso 2):  Introduzca el número: 3428  ->  tres cuatro dos ocho
 *   - Ejemplo (caso 3):  Introduzca el número: 9941  ->  nueve nueve cuatro uno
 * Nota: puedes utilizar bucles while, o no, a tu elección. Eso sí, es necesario utilizar algún tipo de
 * condicional para resolver el problema.
 */
public class ej_ex2_1_Digitos {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int numero, divisor, digito;

        System.out.print("Introduzca el número: ");
        numero = sc.nextInt();

        if (numero < 1000 || numero > 9999) {
            System.out.println("Número no válido.");
        } else {
            // Voy sacando las cifras de izquierda a derecha: miles, centenas, decenas y unidades
            divisor = 1000;
            while (divisor > 0) {
                digito = numero / divisor % 10;

                switch (digito) {
                    case 0: System.out.print("cero "); break;
                    case 1: System.out.print("uno "); break;
                    case 2: System.out.print("dos "); break;
                    case 3: System.out.print("tres "); break;
                    case 4: System.out.print("cuatro "); break;
                    case 5: System.out.print("cinco "); break;
                    case 6: System.out.print("seis "); break;
                    case 7: System.out.print("siete "); break;
                    case 8: System.out.print("ocho "); break;
```

```
                    case 9: System.out.print("nueve "); break;
                }

                divisor = divisor / 10;
            }
            System.out.println();
        }
        sc.close();
    }
}
```

Ejercicio 2. IRPF acumulado

Palabras clave: `#irpf` `#tramos` `#centinela` `#acumulador` `#porcentaje`

Enunciado: leer sueldos entre 10000 y 60000; fuera de rango termina. IRPF 10 % hasta 30000, 15 % hasta 50000, 20 % hasta 60000. Mostrar sueldo, IRPF y totales acumulados. Al salir "Adios".

Idea: centinela con la condición de rango en el `while`; tramos con `if - else if - else`; dos acumuladores.

Código completo · `ej_ex2_2_Irpf.java`

```java
import java.util.Scanner;

/*
 * Ejercicio 2. Realiza en un proyecto Java un programa que realice la siguiente
funcionalidad:
 *  - El programa leerá de teclado números enteros (llamados sueldo) entre el número 10000 y
el
 *    60000, incluidos. Si se introduce un número fuera de este rango, se termina.
 *  - Por cada sueldo introducido, se calculará un porcentaje IRPF que será: el 10% para
números
 *    entre 10000 y 30000, el 15% para números entre el 30001 y el 50000 y el 20% para
números
 *    entre el 50001 y el 60000. Después mostraremos en pantalla el sueldo introducido, el
IRPF
 *    calculado, los sueldos que llevamos acumulados y el IRPF que llevamos acumulados.
 *  - EJEMPLO de ejecución:
 *      Introduzca el sueldo (Entre 10000 y 60000, fuera de rango, salimos):
 *          20500
 *      El sueldo es 20500, el IRPF es el 10% que es 2050.
 *      Totales: 20500 en sueldos y 2050 en IRPF
 *      ...
 *          0
 *      Adios
 */
public class ej_ex2_2_Irpf {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int sueldo, porcentaje, irpf;
        int totalSueldos = 0, totalIrpf = 0;

        System.out.println("Introduzca el sueldo (Entre 10000 y 60000, fuera de rango,
salimos):");
        sueldo = sc.nextInt();

        while (sueldo >= 10000 && sueldo <= 60000) {
            // Elijo el porcentaje según el tramo
            if (sueldo <= 30000) {
                porcentaje = 10;
```

```
            } else if (sueldo <= 50000) {
                porcentaje = 15;
            } else {
                porcentaje = 20;
            }

            irpf = sueldo * porcentaje / 100;
            totalSueldos = totalSueldos + sueldo;
            totalIrpf = totalIrpf + irpf;

            System.out.println("El sueldo es " + sueldo + ", el IRPF es el " + porcentaje +
"% que es " + irpf + ".");
            System.out.println("Totales: " + totalSueldos + " en sueldos y " + totalIrpf + "
en IRPF");

            System.out.println("Introduzca el sueldo (Entre 10000 y 60000, fuera de rango,
salimos):");
            sueldo = sc.nextInt();
        }

        System.out.println("Adios");
        sc.close();
    }
}
```

Ejercicio 3. Cuadrado de números

Palabras clave: `#cuadrado-numeros` `#impar` `#sube-baja`

Enunciado: lado impar (si es par o negativo, volver a pedir). Cuadrado de números que suben hasta el centro y bajan (lado 5 → 12321 / 23432 / 34543 / 23432 / 12321).

Idea: cada número = 1 + lo que sube la fila + lo que sube la columna. "Lo que sube" es la posición hasta la mitad y luego baja: `pos <= lado/2 ? pos : lado - 1 - pos`.

Código completo · `ej_ex2_3_Cuadrado.java`

```
import java.util.Scanner;

/*
 * Ejercicio 3. Realiza en un proyecto Java un programa que realice la siguiente
funcionalidad:
 *  - Queremos, dado un número por teclado, que tiene que ser impar, llamado lado, un
programa
 *    que imprima un cuadrado de números como se muestra a continuación en los ejemplos. Si
 *    se introduce un número par o que sea negativo, se debe de volver a pedir otro hasta que
nos
 *    dan un número impar.
 *  - Ejemplo 1: Lado 3      - Ejemplo 2: Lado 5
 *      121                      12321
 *      232                      23432
 *      121                      34543
 *                               23432
 *                               12321
 * Es un cuadrado que en el centro tiene el número exacto "lado", cada línea tiene tantos
números
 * como dice el "lado", y en las esquinas empiezan en el uno y van incrementándose hasta el
centro,
 * a partir del cual se van decrementando.
 */
public class ej_ex2_3_Cuadrado {

    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
        int lado, subeFila, subeColumna;

        do {
            System.out.print("Introduce el lado (impar y positivo): ");
            lado = sc.nextInt();
            if (lado % 2 == 0 || lado < 0) {
                System.out.println("Número no válido, tiene que ser impar y positivo");
            }
        } while (lado % 2 == 0 || lado < 0);

        for (int fila = 0; fila < lado; fila++) {

            // Lo que suma la fila: sube hasta la mitad y luego baja
            if (fila <= lado / 2) {
                subeFila = fila;
            } else {
                subeFila = lado - 1 - fila;
            }

            for (int columna = 0; columna < lado; columna++) {

                // Lo mismo con la columna
                if (columna <= lado / 2) {
                    subeColumna = columna;
                } else {
                    subeColumna = lado - 1 - columna;
                }

                System.out.print(1 + subeFila + subeColumna);
            }
            System.out.println();
        }
        sc.close();
    }
}
```

Ejercicio 4. Triángulos repetidos

Palabras clave: `#triangulo` `#repetir-figura` `#alto-ancho`

Enunciado: alto y ancho; triángulos de `*` de ancho que bajan de 1 en 1 y se repiten hasta completar `alto` líneas.

Idea: un solo `for` de líneas y un contador de asteriscos que baja; al llegar a 0 vuelve a `ancho`.

Código completo · `ej_ex2_4_Triangulos.java`

```
import java.util.Scanner;

/*
 * Ejercicio 4. Realiza un programa que imprima en pantalla una figura como la siguiente dado
un
 * número por teclado que llamaremos alto y otro ancho:
 * Supongamos alto 8 y ancho 6:
 *     ******
 *     *****
 *     ****
 *     ***
 *     **
 *     *
 *     ******
 *     *****
```

```java
 * Se dibujan triángulos con el símbolo * de ancho x ancho, pero se repiten hasta completar
el
 * número "alto" de líneas máximo. No hay que comprobar la entrada (ancho entre 2 y 10 y alto
 * entre 2 y 20).
 */
public class ej_ex2_4_Triangulos {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int alto, ancho, asteriscos;

        System.out.print("Introduce el alto: ");
        alto = sc.nextInt();

        System.out.print("Introduce el ancho: ");
        ancho = sc.nextInt();

        asteriscos = ancho;
        for (int linea = 1; linea <= alto; linea++) {
            for (int i = 0; i < asteriscos; i++) {
                System.out.print("*");
            }
            System.out.println();

            // Quito un asterisco y si llego a 0 empiezo otro triángulo
            asteriscos--;
            if (asteriscos == 0) {
                asteriscos = ancho;
            }
        }
        sc.close();
    }
}
```

Parte 4. Otros ejercicios de clase

Triaje hospitalario

Palabras clave: `#triaje` `#tramos` `#boolean` `#colores`

Enunciado: según los síntomas, triaje ROJO (compromiso vital inminente: hemorragia, ictus, arritmia, fc < 40 o > 110, fr < 10 o > 20), AMARILLO (tumor, neumonía, infección generalizada) o VERDE.

Idea: se pregunta **de lo más grave a lo menos**: un `if` con todas las condiciones de rojo unidas por `||`, luego `else if` para amarillo y `else` verde. Los síntomas son `boolean` leídos con `nextBoolean()` (se responde `true` o `false`).

Código completo · `Triaje.java`

```java
import java.util.Scanner;

public class Triaje {

    public static void main(String[] args) {

        //Primero: reglas
        // Rojo: gravedad inminente de compromiso vital: hemorragia, h interna, ictus, cv: arritmia, fc baja, fc alta, fr baja o alta
        // Amarillo: Gravedad pero compromiso vital a futuro: cancer?, neumonia?, infeccion generalizada
        // Verde: No hay compromiso vital a presente o futuro

        //entrada
        // ¿que pedimos al medico? hemorragia(boolean), interna(boolean). ictus(boolean), arritmia(boolean)
        // fcbaja es cuando fc<40, fcalta es fc>110 en reposo, frbaja est por debajo de 10/minuto, fralta es fr>20
        // tumor(boolean), neumonia(boolean), igeneral(boolean)

        //Memoria
        Scanner sc = new Scanner(System.in);

        boolean hemorragia, interna, ictus, arritmia;
        boolean tumor, neumonia, igeneral;
        int fc, fr;
        String color;

        // Pedimos los datos al medico (true = si, false = no)
        System.out.println("TRIAJE HOSPITALARIO (responde true o false)");

        System.out.print("¿Tiene hemorragia? ");
        hemorragia = sc.nextBoolean();

        System.out.print("¿La hemorragia es interna? ");
        interna = sc.nextBoolean();

        System.out.print("¿Tiene un ictus? ");
        ictus = sc.nextBoolean();

        System.out.print("¿Tiene arritmia? ");
        arritmia = sc.nextBoolean();
```

```
        System.out.print("Frecuencia cardiaca en reposo (pulsaciones/min): ");
        fc = sc.nextInt();

        System.out.print("Frecuencia respiratoria (respiraciones/min): ");
        fr = sc.nextInt();

        System.out.print("¿Tiene un tumor? ");
        tumor = sc.nextBoolean();

        System.out.print("¿Tiene neumonia? ");
        neumonia = sc.nextBoolean();

        System.out.print("¿Tiene una infeccion generalizada? ");
        igeneral = sc.nextBoolean();

        //Proceso
        // Primero miramos lo mas grave (rojo), si no, amarillo y si no, verde
        if (hemorragia || interna || ictus || arritmia || fc < 40 || fc > 110 || fr < 10 ||
fr > 20) {
            color = "ROJO";
        } else if (tumor || neumonia || igeneral) {
            color = "AMARILLO";
        } else {
            color = "VERDE";
        }

        //salida
        System.out.println("El paciente tiene triaje " + color);

        if (color.equals("ROJO")) {
            System.out.println("Compromiso vital inminente, atender YA");
        } else if (color.equals("AMARILLO")) {
            System.out.println("Es grave pero el compromiso vital es a futuro");
        } else {
            System.out.println("No hay compromiso vital ni ahora ni a futuro");
        }

        sc.close();
    }
}
```

Juego de los dados

Palabras clave: `#dados` `#jugadores` `#empate` `#ganador` `#desempate`

Reglas: gana quien saque más 6; si empatan en seises, quien sume más; si no, empate. Cada jugador mete sus 2 dados.

Dos jugadores

Idea: cada jugador: 2 `nextInt()`, suma y contar seises con dos `if`. Luego `if - else if` en el orden de las reglas: seises, suma, empate.

Código completo · `JuegoDados.java`

```
import java.util.Scanner;

public class JuegoDados {

    public static void main(String[] args) {
```

```java
        //Reglas
        //El jugador que mas 6 saque gana
        //Si tienen los mismo "numeros 6", el que sume una tirada mayor gana
        //En otro caso empate

        //Memoria
        Scanner sc = new Scanner(System.in);
        int dado1, dado2;
        int seis1 = 0, seis2 = 0;
        int suma1, suma2;

        System.out.println("=== JUEGO DE LOS DADOS (2 jugadores) ===");

        //Entrada jugador 1
        do {
            System.out.print("Jugador 1, primer dado: ");
            dado1 = sc.nextInt();
        } while (dado1 < 1 || dado1 > 6);

        do {
            System.out.print("Jugador 1, segundo dado: ");
            dado2 = sc.nextInt();
        } while (dado2 < 1 || dado2 > 6);

        suma1 = dado1 + dado2;
        if (dado1 == 6) {
            seis1++;
        }
        if (dado2 == 6) {
            seis1++;
        }

        //Entrada jugador 2
        do {
            System.out.print("Jugador 2, primer dado: ");
            dado1 = sc.nextInt();
        } while (dado1 < 1 || dado1 > 6);

        do {
            System.out.print("Jugador 2, segundo dado: ");
            dado2 = sc.nextInt();
        } while (dado2 < 1 || dado2 > 6);

        suma2 = dado1 + dado2;
        if (dado1 == 6) {
            seis2++;
        }
        if (dado2 == 6) {
            seis2++;
        }

        //Salida
        System.out.println();
        System.out.println("Jugador 1: " + seis1 + " seises y suma " + suma1);
        System.out.println("Jugador 2: " + seis2 + " seises y suma " + suma2);

        //Proceso: primero los 6 y si están iguales la suma
        if (seis1 > seis2) {
            System.out.println("Gana el jugador 1 porque tiene más seises");
        } else if (seis2 > seis1) {
            System.out.println("Gana el jugador 2 porque tiene más seises");
        } else if (suma1 > suma2) {
            System.out.println("Tienen los mismos seises, gana el jugador 1 porque suma
```

```
más");
        } else if (suma2 > suma1) {
            System.out.println("Tienen los mismos seises, gana el jugador 2 porque suma
más");
        } else {
            System.out.println("Empate");
        }

        sc.close();
    }
}
```

Tres jugadores

Idea: 1) máximo de seises; 2) entre los que lo tienen, la suma más alta; 3) contar cuántos tienen las dos cosas: si es uno, gana; si son más, empate.

Código completo · `JuegoDados3.java`

```java
import java.util.Scanner;

public class JuegoDados3 {

    public static void main(String[] args) {

        //Reglas
        //El jugador que mas 6 saque gana
        //Si tienen los mismo "numeros 6", el que sume una tirada mayor gana
        //En otro caso empate

        //Memoria
        Scanner sc = new Scanner(System.in);
        int dado1, dado2;
        int seis1 = 0, seis2 = 0, seis3 = 0;
        int suma1, suma2, suma3;
        int maxSeis, maxSuma;
        int ganadores = 0, ganador = 0;

        System.out.println("=== JUEGO DE LOS DADOS (3 jugadores) ===");

        //Entrada jugador 1
        do {
            System.out.print("Jugador 1, primer dado: ");
            dado1 = sc.nextInt();
        } while (dado1 < 1 || dado1 > 6);

        do {
            System.out.print("Jugador 1, segundo dado: ");
            dado2 = sc.nextInt();
        } while (dado2 < 1 || dado2 > 6);

        suma1 = dado1 + dado2;
        if (dado1 == 6) {
            seis1++;
        }
        if (dado2 == 6) {
            seis1++;
        }

        //Entrada jugador 2
        do {
            System.out.print("Jugador 2, primer dado: ");
```

```
            dado1 = sc.nextInt();
        } while (dado1 < 1 || dado1 > 6);

        do {
            System.out.print("Jugador 2, segundo dado: ");
            dado2 = sc.nextInt();
        } while (dado2 < 1 || dado2 > 6);

        suma2 = dado1 + dado2;
        if (dado1 == 6) {
            seis2++;
        }
        if (dado2 == 6) {
            seis2++;
        }

        //Entrada jugador 3
        do {
            System.out.print("Jugador 3, primer dado: ");
            dado1 = sc.nextInt();
        } while (dado1 < 1 || dado1 > 6);

        do {
            System.out.print("Jugador 3, segundo dado: ");
            dado2 = sc.nextInt();
        } while (dado2 < 1 || dado2 > 6);

        suma3 = dado1 + dado2;
        if (dado1 == 6) {
            seis3++;
        }
        if (dado2 == 6) {
            seis3++;
        }

        //Proceso
        //Primero miro cual es el maximo de seises
        maxSeis = seis1;
        if (seis2 > maxSeis) {
            maxSeis = seis2;
        }
        if (seis3 > maxSeis) {
            maxSeis = seis3;
        }

        //De los que tienen el maximo de seises, miro la suma mas alta
        maxSuma = 0;
        if (seis1 == maxSeis && suma1 > maxSuma) {
            maxSuma = suma1;
        }
        if (seis2 == maxSeis && suma2 > maxSuma) {
            maxSuma = suma2;
        }
        if (seis3 == maxSeis && suma3 > maxSuma) {
            maxSuma = suma3;
        }

        //Cuento cuantos jugadores tienen las dos cosas, si es mas de uno es empate
        if (seis1 == maxSeis && suma1 == maxSuma) {
            ganadores++;
            ganador = 1;
        }
        if (seis2 == maxSeis && suma2 == maxSuma) {
```

```
            ganadores++;
            ganador = 2;
        }
        if (seis3 == maxSeis && suma3 == maxSuma) {
            ganadores++;
            ganador = 3;
        }

        //Salida
        System.out.println();
        System.out.println("Jugador 1: " + seis1 + " seises y suma " + suma1);
        System.out.println("Jugador 2: " + seis2 + " seises y suma " + suma2);
        System.out.println("Jugador 3: " + seis3 + " seises y suma " + suma3);

        if (ganadores == 1) {
            System.out.println("Gana el jugador " + ganador);
        } else {
            System.out.println("Empate entre " + ganadores + " jugadores");
        }

        sc.close();
    }
}
```

N jugadores

Idea: un `for` por jugador y comparar solo con **el que va ganando**: si lo supera, nuevo líder; si es igual, `empate = true`.

Código completo · `JuegoDadosN.java`

```
import java.util.Scanner;

public class JuegoDadosN {

    public static void main(String[] args) {

        //Reglas
        //El jugador que mas 6 saque gana
        //Si tienen los mismo "numeros 6", el que sume una tirada mayor gana
        //En otro caso empate

        //Memoria
        Scanner sc = new Scanner(System.in);
        int jugadores, dado1, dado2, seis, suma;
        int maxSeis = -1, maxSuma = -1, ganador = 0;
        boolean empate = false;

        System.out.println("=== JUEGO DE LOS DADOS ===");

        //Entrada: cuantos juegan (minimo 2)
        do {
            System.out.print("¿Cuántos jugadores vais a jugar? ");
            jugadores = sc.nextInt();
            if (jugadores < 2) {
                System.out.println("Tienen que jugar al menos 2");
            }
        } while (jugadores < 2);

        //Proceso: cada jugador tira y lo comparo con el que va ganando
        for (int j = 1; j <= jugadores; j++) {
```

```
            do {
                System.out.print("Jugador " + j + ", primer dado: ");
                dado1 = sc.nextInt();
            } while (dado1 < 1 || dado1 > 6);

            do {
                System.out.print("Jugador " + j + ", segundo dado: ");
                dado2 = sc.nextInt();
            } while (dado2 < 1 || dado2 > 6);

            suma = dado1 + dado2;
            seis = 0;
            if (dado1 == 6) {
                seis++;
            }
            if (dado2 == 6) {
                seis++;
            }

            System.out.println("Jugador " + j + ": " + seis + " seises y suma " + suma);

            //Si tiene mas seises, o los mismos pero suma mas, pasa a ir ganando
            if (seis > maxSeis || (seis == maxSeis && suma > maxSuma)) {
                maxSeis = seis;
                maxSuma = suma;
                ganador = j;
                empate = false;
            } else if (seis == maxSeis && suma == maxSuma) {
                //Si tiene exactamente lo mismo que el que va ganando, hay empate
                empate = true;
            }
        }

        //Salida
        System.out.println();
        if (empate) {
            System.out.println("Empate con " + maxSeis + " seises y suma " + maxSuma);
        } else {
            System.out.println("Gana el jugador " + ganador + " con " + maxSeis + " seises y
suma " + maxSuma);
        }

        sc.close();
    }
}
```

Árboles de Navidad

Palabras clave: `#arbol` `#navidad` `#figura`

En la fila f (desde 0): `alto - 1 - f` de relleno y `f · 2 + 1` asteriscos (1, 3, 5…). El tronco va centrado con `alto - 1` de relleno.

Con puntos y tronco de asteriscos

Código completo · `ArbolNavidad.java`

```
import java.util.Scanner;

public class ArbolNavidad {
```

```
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int altura, cantidad;

        System.out.print("Introduce la altura del árbol: ");
        altura = sc.nextInt();

        System.out.print("Introduce la cantidad: ");
        cantidad = sc.nextInt();

        for (int arbol = 1; arbol <= cantidad; arbol++) {

            // Copa: en cada línea un punto menos a cada lado y 2 asteriscos más
            for (int fila = 0; fila < altura; fila++) {
                for (int i = 0; i < altura - 1 - fila; i++) {
                    System.out.print(".");
                }
                for (int i = 0; i < fila * 2 + 1; i++) {
                    System.out.print("*");
                }
                for (int i = 0; i < altura - 1 - fila; i++) {
                    System.out.print(".");
                }
                System.out.println();
            }

            // Tronco: 2 líneas con un asterisco en el centro
            for (int fila = 0; fila < 2; fila++) {
                for (int i = 0; i < altura - 1; i++) {
                    System.out.print(".");
                }
                System.out.print("*");
                for (int i = 0; i < altura - 1; i++) {
                    System.out.print(".");
                }
                System.out.println();
            }

            // Línea en blanco entre árboles (no después del último)
            if (arbol < cantidad) {
                System.out.println();
            }
        }
        sc.close();
    }
}
```

Con tronco `|`

Código completo · `ArbolNavidad2.java`

```
import java.util.Scanner;

public class ArbolNavidad2 {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int altura, cantidad;

        System.out.print("Introduce la altura del árbol: ");
        altura = sc.nextInt();
```

```
        System.out.print("Introduce la cantidad: ");
        cantidad = sc.nextInt();

        for (int arbol = 1; arbol <= cantidad; arbol++) {

            // Copa: en cada línea un punto menos a cada lado y 2 asteriscos más
            for (int fila = 0; fila < altura; fila++) {
                for (int i = 0; i < altura - 1 - fila; i++) {
                    System.out.print(".");
                }
                for (int i = 0; i < fila * 2 + 1; i++) {
                    System.out.print("*");
                }
                for (int i = 0; i < altura - 1 - fila; i++) {
                    System.out.print(".");
                }
                System.out.println();
            }

            // Tronco: 2 líneas con una barra | en el centro
            for (int fila = 0; fila < 2; fila++) {
                for (int i = 0; i < altura - 1; i++) {
                    System.out.print(".");
                }
                System.out.print("|");
                for (int i = 0; i < altura - 1; i++) {
                    System.out.print(".");
                }
                System.out.println();
            }

            // Línea en blanco entre árboles (no después del último)
            if (arbol < cantidad) {
                System.out.println();
            }
        }
        sc.close();
    }
}
```

Con bolas aleatorias

Palabras clave: `#bolas` `#aleatorio`

`if (Math.random() < 0.3)` pinta un `0` (bola) más o menos 1 de cada 3 veces.

Código completo · `ArbolNavidad3.java`

```
import java.util.Scanner;

public class ArbolNavidad3 {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int altura;

        System.out.print("Introduce la altura del árbol: ");
        altura = sc.nextInt();

        // Copa: espacios a la izquierda y cada fila con 2 caracteres más
        for (int fila = 0; fila < altura; fila++) {
            for (int i = 0; i < altura - 1 - fila; i++) {
```

```
                System.out.print(" ");
            }
            for (int i = 0; i < fila * 2 + 1; i++) {
                // La punta siempre es un asterisco
                if (fila == 0) {
                    System.out.print("*");
                // Si no, más o menos 1 de cada 3 es una bola
                } else if (Math.random() < 0.3) {
                    System.out.print("0");
                } else {
                    System.out.print("*");
                }
            }
            System.out.println();
        }

        // Tronco: 2 líneas con una barra en el centro
        for (int fila = 0; fila < 2; fila++) {
            for (int i = 0; i < altura - 1; i++) {
                System.out.print(" ");
            }
            System.out.println("|");
        }
        sc.close();
    }
}
```

Hecho con while

Palabras clave: `#sin-for` `#while`

Cada `for` pasa a `while`: contador a 0 antes, condición en el `while` e `i++` al final de dentro.

Código completo · `ArbolNavidad4.java`

```
import java.util.Scanner;

public class ArbolNavidad4 {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int altura, fila, i;

        System.out.print("Introduce la altura del árbol: ");
        altura = sc.nextInt();

        // Copa: espacios a la izquierda y cada fila con 2 caracteres más
        fila = 0;
        while (fila < altura) {

            i = 0;
            while (i < altura - 1 - fila) {
                System.out.print(" ");
                i++;
            }

            i = 0;
            while (i < fila * 2 + 1) {
                // La punta siempre es un asterisco
                if (fila == 0) {
                    System.out.print("*");
                // Si no, más o menos 1 de cada 3 es una bola
                } else if (Math.random() < 0.3) {
```

```
                    System.out.print("0");
                } else {
                    System.out.print("*");
                }
                i++;
            }

            System.out.println();
            fila++;
        }

        // Tronco: 2 líneas con una barra en el centro
        fila = 0;
        while (fila < 2) {
            i = 0;
            while (i < altura - 1) {
                System.out.print(" ");
                i++;
            }
            System.out.println("|");
            fila++;
        }
        sc.close();
    }
}
```

Con / \ y base ^^^[_]^^^

Palabras clave: `#barra-invertida`

La barra `\` se escribe `"\\"` porque sola es un carácter especial.

Código completo · `ArbolNavidad5.java`

```
import java.util.Scanner;

public class ArbolNavidad5 {

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int altura;

        System.out.print("Introduce la altura del árbol: ");
        altura = sc.nextInt();

        // Estrella de arriba
        for (int i = 0; i < altura; i++) {
            System.out.print(" ");
        }
        System.out.println("*");

        // Copa: cada fila es / + puntos y bolas + \
        for (int fila = 0; fila < altura; fila++) {
            for (int i = 0; i < altura - 1 - fila; i++) {
                System.out.print(" ");
            }
            System.out.print("/");
            for (int i = 0; i < fila * 2 + 1; i++) {
                // Más o menos 1 de cada 4 es una bola
                if (Math.random() < 0.25) {
                    System.out.print("O");
                } else {
                    System.out.print(".");
```

```
                }
            }
            System.out.println("\\"); // la barra \ se escribe doble porque es especial
        }

        // Base: césped ^ con el tronco [_] en el centro
        for (int i = 0; i < altura - 1; i++) {
            System.out.print("^");
        }
        System.out.print("[_]");
        for (int i = 0; i < altura - 1; i++) {
            System.out.print("^");
        }
        System.out.println();

        sc.close();
    }
}
```